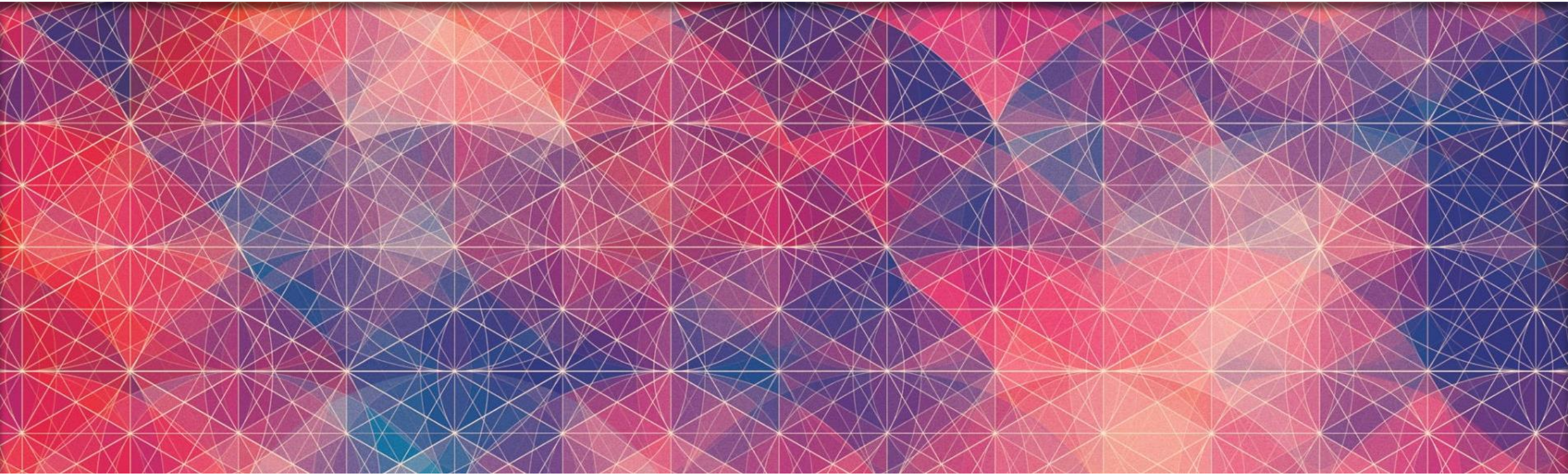
Universität St.Gallen



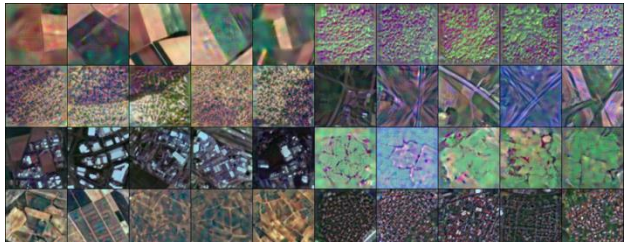
Weight Symmetrie Workshop - ICML 2026

Weight Space Learning: Learning from Populations of Neural Networks

Damian Borth

Where is the information / knowledge stored?

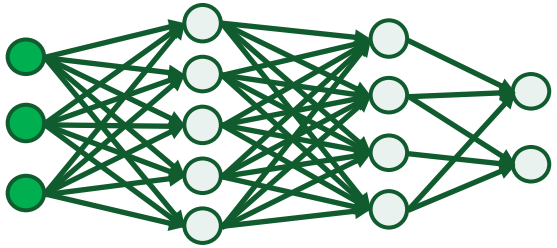
Dataset



Questions:
How much information is in the data? Is this information similar? Can we synthesize / compress it?

For example:
- Dataset similarity
- Dataset Distillation
- Training on Synthetized Data

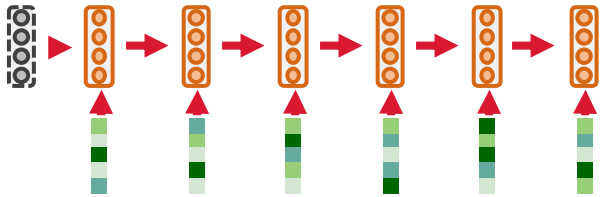
Model Parameter



Questions:
What can we infer from the model parameter? Do we need all of these parameters?

For example:
- Inference of latent factors
- Model Compression
- Knowledge Distillation

Context

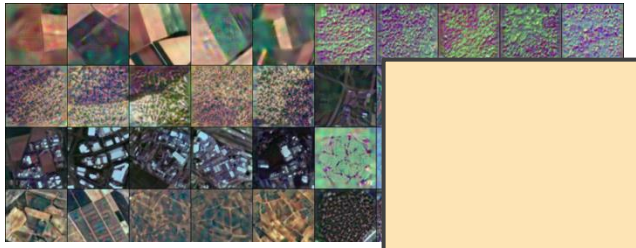


Questions:
Can we condition the forward inference by an external input and memorize it through inference?

For example:
- Hidden state (RNNs)
- Context Vector (Attention)
- Prompt (LLMs)

Where is the information / knowledge stored?

Dataset



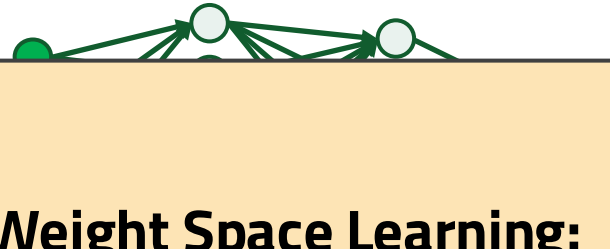
Questions:

How much informatio
data? Is this informat
Can we synthesize / c

For example:

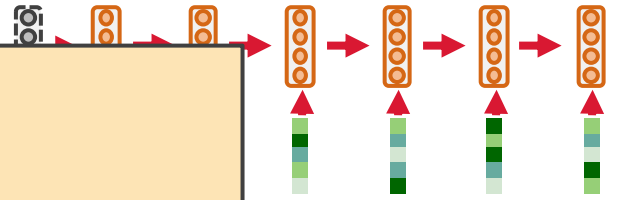
- Dataset similarit
- Dataset Distillation
- Training on Synthetized Data

Model Parameter



- Model Compression
- Knowledge Distillation

Context



ition the forward
an external input and
through inference?

:

state (RNNs)

- Context Vector (Attention)
- Prompt (LLMs)

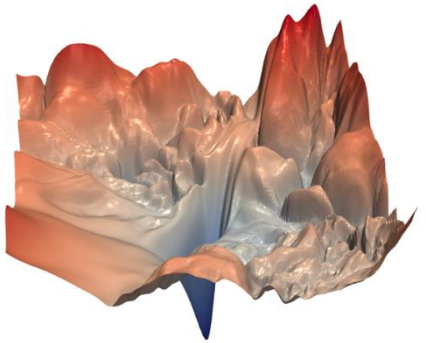
Weight Space Learning:

Learning representations of neural network model parameters (weights & biases) to be exploited for multiple downstream tasks

Weight Space Learning: Thinking of Neural Network Weights as new Data Modality

Challenges for Neural Network Training

Neural Networks are successfully applied on multiple domains



Li et al.; NeurIPS 2018; *Visualizing the Loss Landscape of Neural Nets*

Loss surface and optimization problem of Neural Networks are non-convex

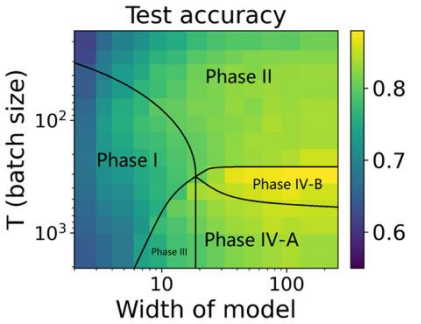
Goodfellow, Vinyals, Saxe; ICLR 2015; *Qualitatively characterizing neural network optimization problems*
Dauphin et al.; NeurIPS 2014; *Identifying and attacking the saddle point problem in high-dimensional non-convex optimization*
LeCun, Bengion, Hinton; Nature 2015; *Deep Learning*

Neural Network training optimization is high dimensional

Brown et al.; 2020; *Language Models are Few-Shot Learners*
Larsen et al.; ICML 2021; *How many degrees of freedom do we need to train deep networks: a loss landscape perspective*

Neural Network training is sensitive to hyperparameters and random initialization

F Hutter, L Kotthoff, J Vanschoren, Springer, *Automated machine learning: methods, systems, challenges*
Hanin, Rolnick; NeurIPS 2018; *How to Start Training: The Effect of Initialization and Architecture*
Yang et al.; NeurIPS 2021; *Taxonomizing local versus global structure in neural network loss landscapes*

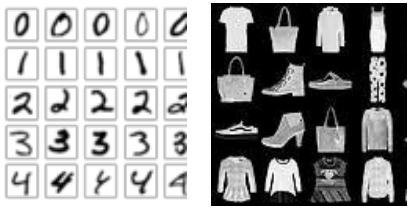


Yang et al.; NeurIPS 2021; Taxonomizing local versus global structure in neural network loss landscapes

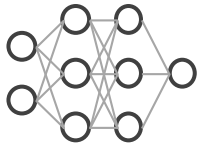
We want to better understand the relation between characteristics (behaviour, performance, latent generating factors) of Neural Network models and their solution in weight space

Learning from Populations of NN Models

Dataset / Task



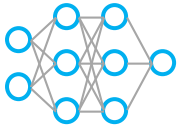
Architecture



Hyperparamerters

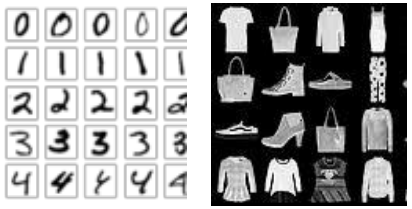
- Optimizer
- Activation
- Initialization Method
- Learning Rate
- Regularization (L1/L2)

Model

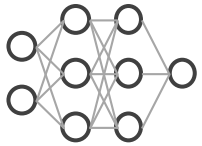


Learning from Populations of NN Models

Dataset / Task



Architecture



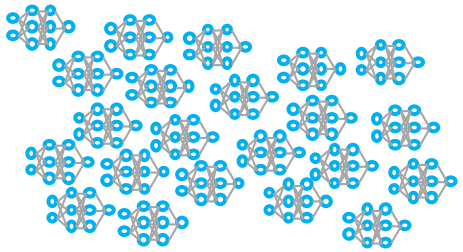
Hyperparamerters

- Optimizer
- Activation
- Initialization Method
- Learning Rate
- Regularization (L1/L2)

Hypothesis:

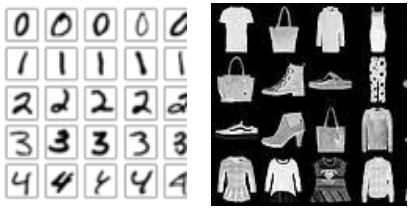
1. Neural Networks populate a structure in weight space
2. That structure contains information on properties and generating factors of the models

Model Population

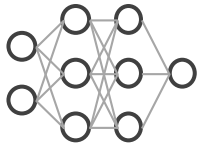


Learning from Populations of NN Models

Dataset / Task



Architecture



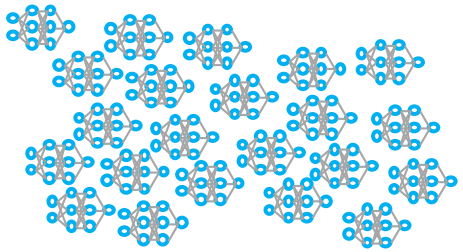
Hyperparamerters

- Optimizer
- Activation
- Initialization Method
- Learning Rate
- Regularization (L1/L2)

Hypothesis:

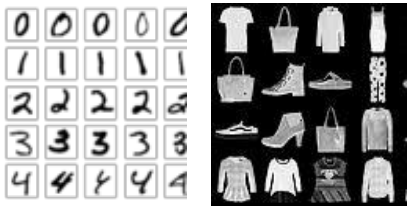
1. Neural Networks populate a structure in weight space
2. That structure contains information on properties and generating factors of the models

Weight Space θ

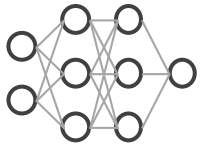


Learning from Populations of NN Models

Dataset / Task



Architecture

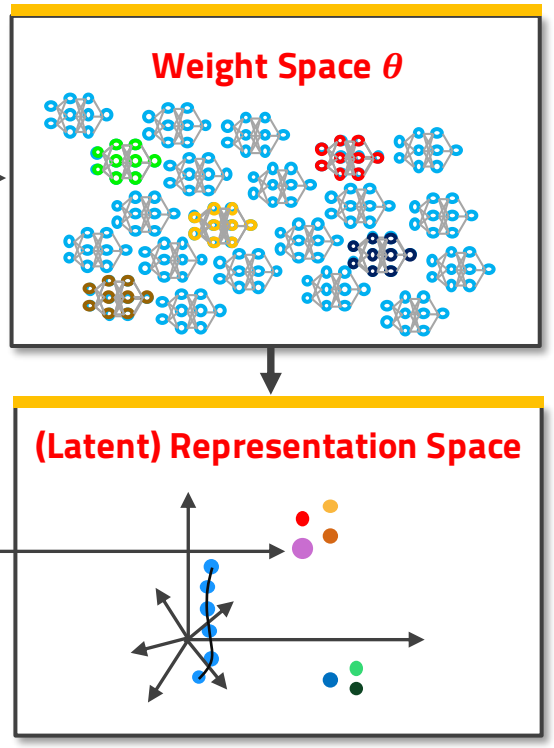


Hyperparamerters

- Optimizer
- Activation
- Initialization Method
- Learning Rate
- Regularization (L1/L2)

Hypothesis:

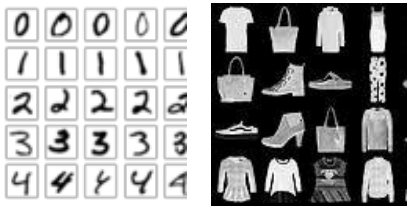
1. Neural Networks populate a structure in weight space
2. That structure contains information on properties and generating factors of the models



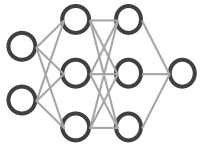
Goal: Learn meaningful and rich representations of populations of Neural Network models

Learning from Populations of NN Models

Dataset / Task



Architecture

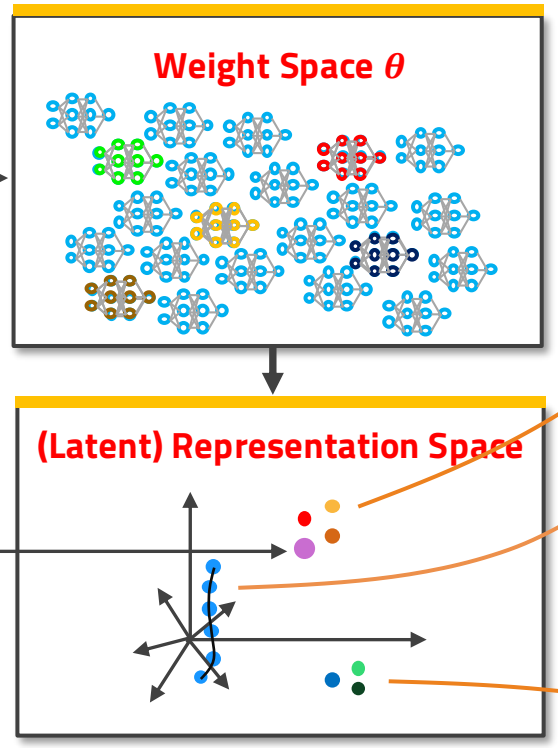


Hyperparamerters

- Optimizer
- Activation
- Initialization Method
- Learning Rate
- Regularization (L1/L2)

Hypothesis:

1. Neural Networks populate a structure in weight space
2. That structure contains information on properties and generating factors of the models



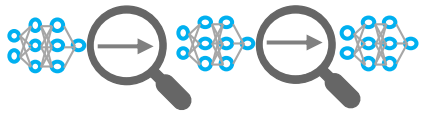
Goal: Learn meaningful and rich representations of populations of Neural Network models

Model Analysis



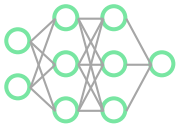
diagnostics, accuracy, versioning, ...

Learning Dynamics



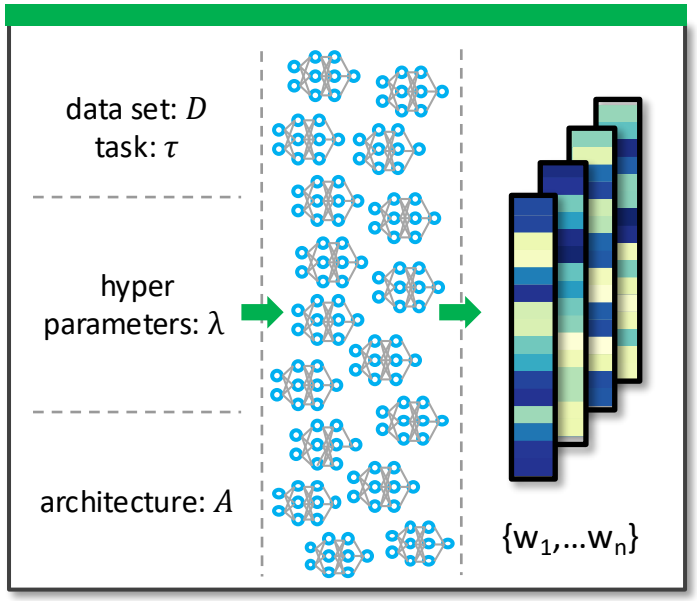
early-stopping, optimization, ...

Model Generation

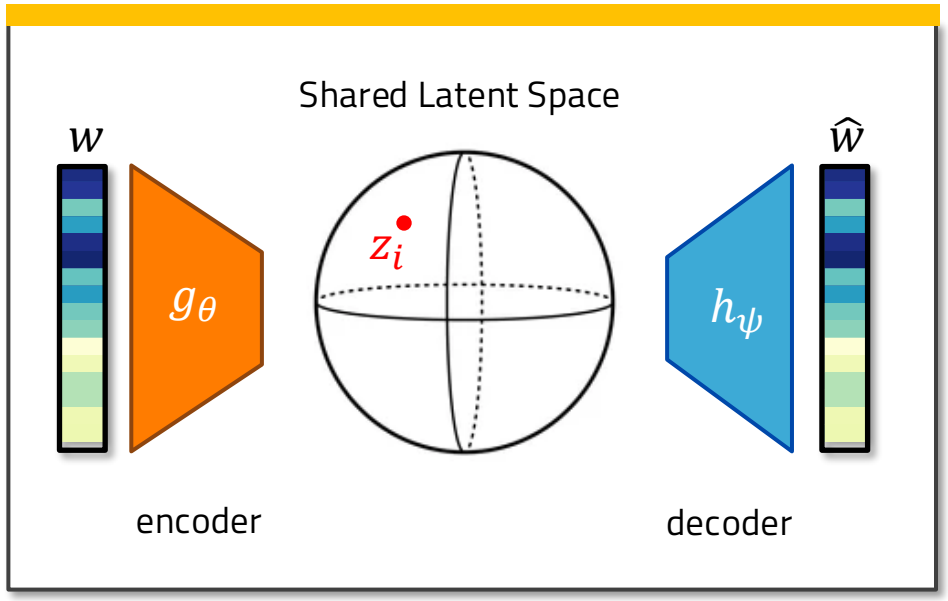


initialization, transfer-learning, meta-learning, sparsification ...

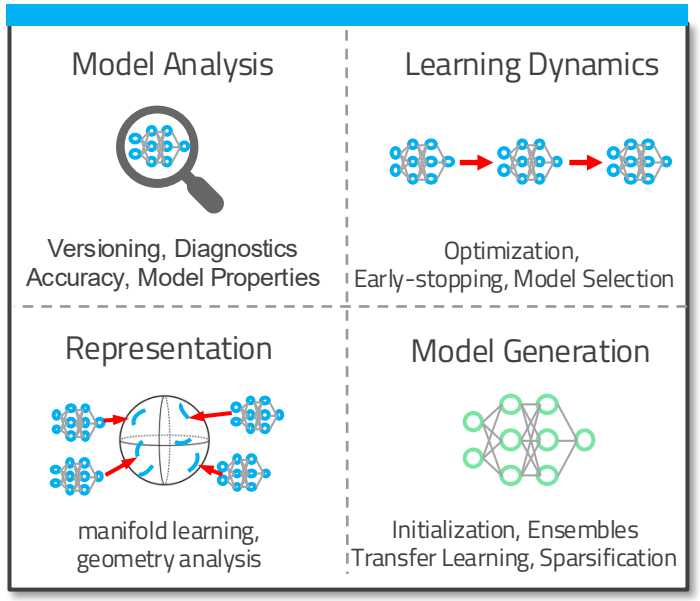
Weight Space Learning Setup



(A) Model Zoos

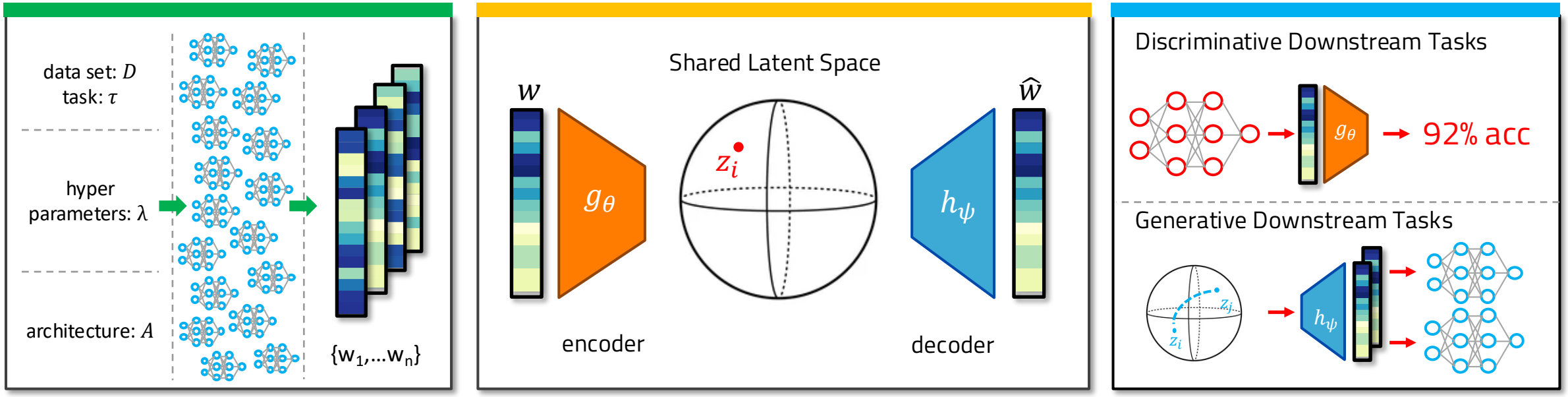


(B) Representation Learning Backbone



(C) Downstream Tasks

Weight Space Learning Setup



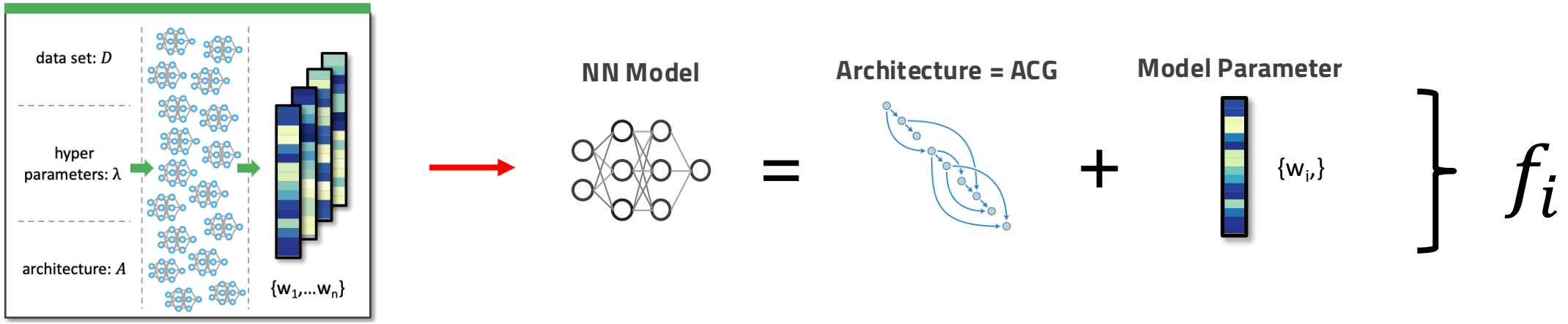
(A) Model Zoos (B) Representation Learning Backbone (C) Downstream Tasks

We "simply" do with **weights** what people have done with **words** and **pixels**.

Instead of **words** or **pixels**, we have **weights**.
Instead of **sentences** or **patches**, we have **layers**.

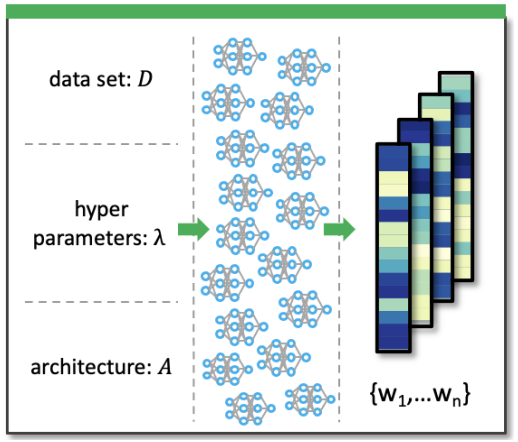
Weight Space Symmetries & Permutations

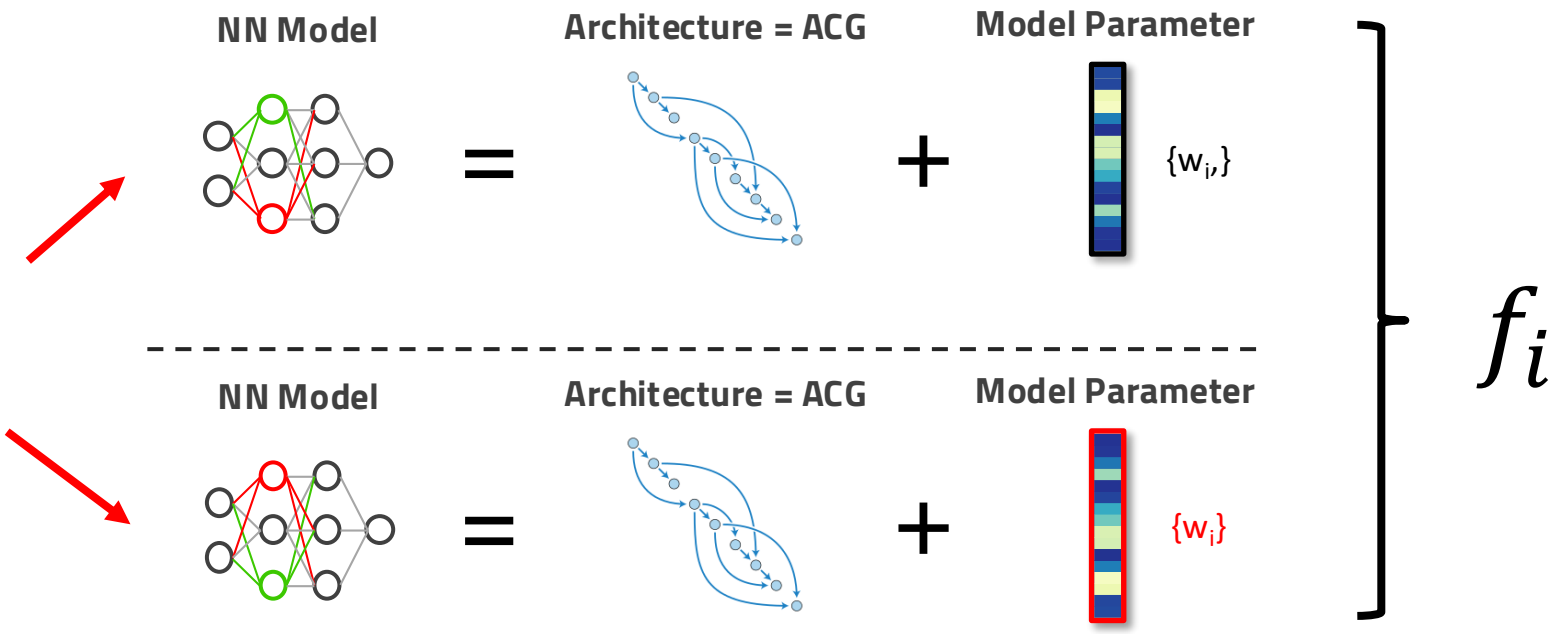
Neural Network Model vs. Weight Vector



Weight Space Symmetries & Permutations

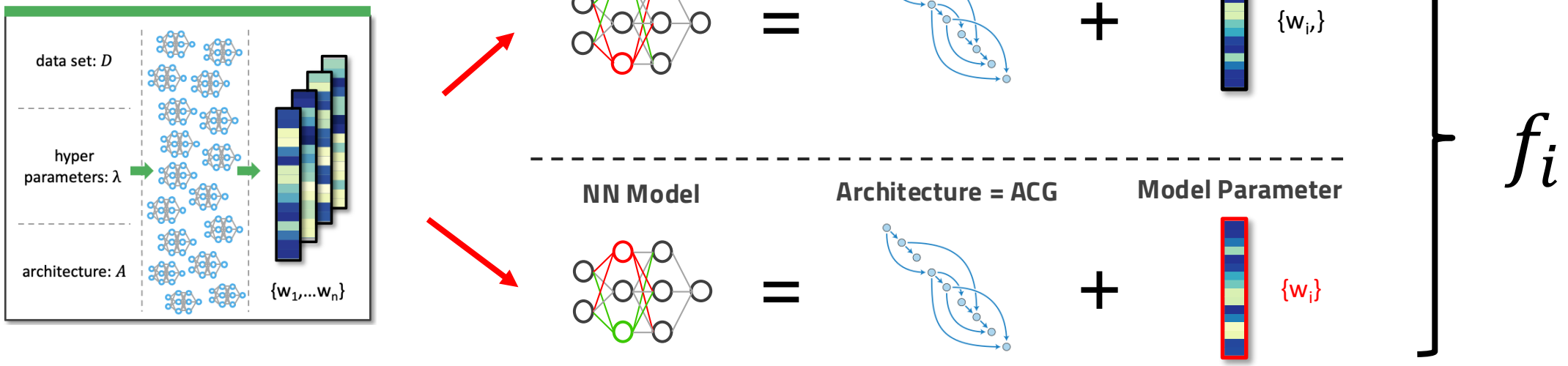
Neural Network Model vs. Weight Vector





Weight Space Symmetries & Permutations

Neural Network Model vs. Weight Vector



How can we learn such representations? How important are symmetries?
How can we perform augmentation? What is important for learning at all?

-> for more on augmentations look up: Deep Augmentations in Deep Weight Spaces, 2023 / 2024

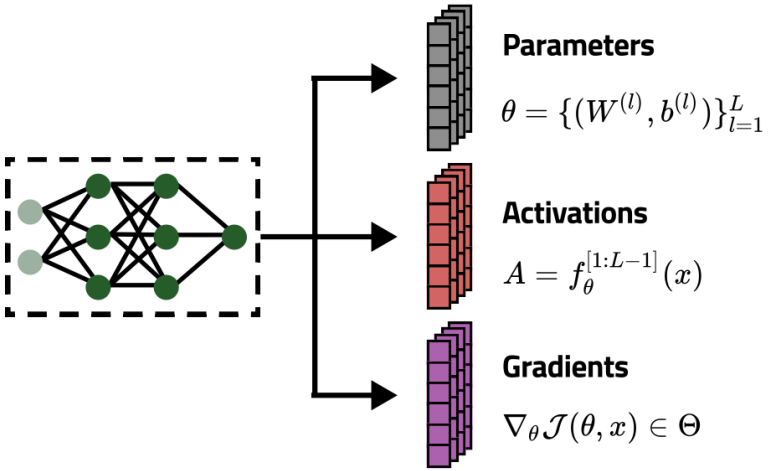
Overview of Weight Space Learning Work

Conceptual View

Types of Data Obtained from Trained Neural Networks

As per Herrmann et al. [M1] there is the **Mechanistic vs. Functionalist** approach

Extension for using **Training Dynamics** as data for learning



Parameters

$\theta = \{(W^{(l)}, b^{(l)})\}_{l=1}^{L}$

Activations

$A = f_\theta^{[1:L-1]}(x)$

Gradients

$\nabla_\theta \mathcal{J}(\theta, x) \in \Theta$

HIGH ← **Availability** → **LOW**

Mechanistic

PyTorch Tensor File (*.pt)
A state dictionary mapping keys to valued tensors

Model Parameters
Set of weights & biases at each layer

$\theta = \{(W^{(l)}, b^{(l)})\}_{l=1}^{L}$

Weight Space

$\Theta := \prod_{l=1}^{L} \left(\mathcal{W}^{(l)} \times \mathcal{B}^{(l)}\right)$

The space of weights & biases:

$W^{(l)} \in \mathcal{W}^{(l)} := \mathbb{R}^{d_l \times d_{l-1}}$

$b^{(l)} \in \mathcal{B}^{(l)} := \mathbb{R}^{d_l}$

Only requires the model file (*.pt) containing the raw parameter values of the neural network model.

Functionalist

Functional Class (*.py)
Defines the computational graph mapping input to output

Intermediate Activations
Set of activations at each layer

$A = f_\theta^{[1:L-1]}(x) = \{a^{(l)}(a^{(l-1)})\}_{l=1}^{L-1}$

Activation Space

$\mathcal{A} := \prod_{l=1}^{L-1} \mathcal{A}^l$

The space intermediate activations:

$a^{(l)} \in \mathcal{A}^{(l)} := \mathbb{R}^{d_l}$

Requires the functional model class (*.py) and a set of probes (inputs) for computing the activations.

Training Dynamics

Training Algorithm
Defines how gradients and momentum are computed

Computed Gradients

$\nabla_\theta \mathcal{J}(\theta, x) \in \Theta$

$\nabla_{W^{(l)}} \mathcal{J}(\theta, x) \in \mathcal{W}^{(l)}$

$\nabla_{b^{(l)}} \mathcal{J}(\theta, x) \in \mathcal{B}^{(l)}$

Momentum Terms
Computes the "velocity" of updates

$m_t = \gamma m_{t-1} + \nabla_{\theta^t} \mathcal{J}(\theta^t, x)$

$m_t \in \Theta, \gamma \in [0, 1)$

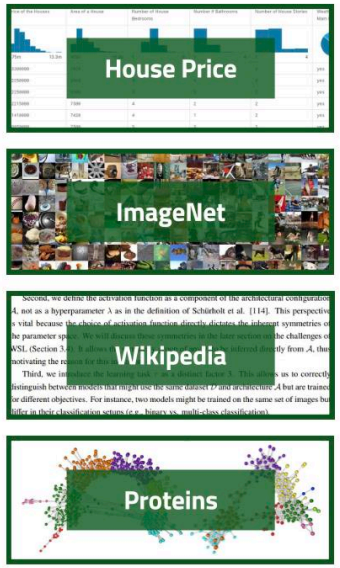
Can be contained in the model file (*.pt) if the parameters are exported with the optimizer state.

Slides from Tobias Ettling

Model Population View

$\mathcal{D}$

Dataset

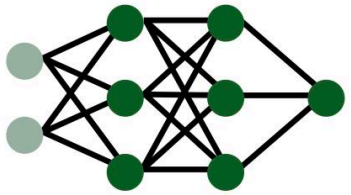


Train/Test Split | #Samples

Specifies the data domain and sampling distribution used to train and evaluate a model.

$\mathcal{A}$

Architecture



$$f_\theta : \mathcal{X} \to \mathcal{Y}$$

$$\mathcal{X} = \mathbb{R}^{d_1 \times \cdots \times d_r}$$

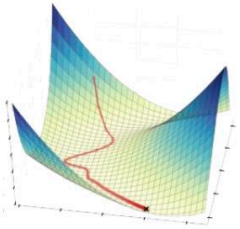
$$\mathcal{Y} = \mathbb{R}^{d_1 \times \cdots \times d_k}$$

Activation Function | #Layers
Architecture Type | Kernel Size
Batch Normalization | Pooling
Skip-Connections | ...

Defines the computational graph, shape/number of parameters and their symmetry group.

λ

Hyper-parameters



Gradient-based Optimization Algorithm
Backpropagation:

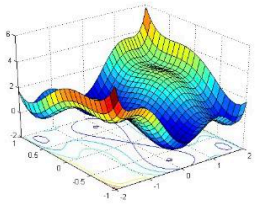
$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_\theta \mathcal{J}(\theta^{(t)})$$

Optimizer | Batch-Size | Epochs
Dropout | Learning Rate | Seeds
Initialization | Schedulers | ...

Defines how the parameters are initialized and updated during the training algorithm.

τ

Learning Task



$$f_\tau : X \to Y$$

$$X \subset \mathcal{X}, Y \subset \mathcal{Y}$$

Loss Function

$\mathcal{L}(f_\theta, x)$

Measures Model Performance

Defines the specific task to perform on a dataset along withe the output behaviour and loss function.

$\mathcal{E}$

Stochastic Component



Initialization Method (e.g. He)

Random mini-batch sampling (e.g. SGD)

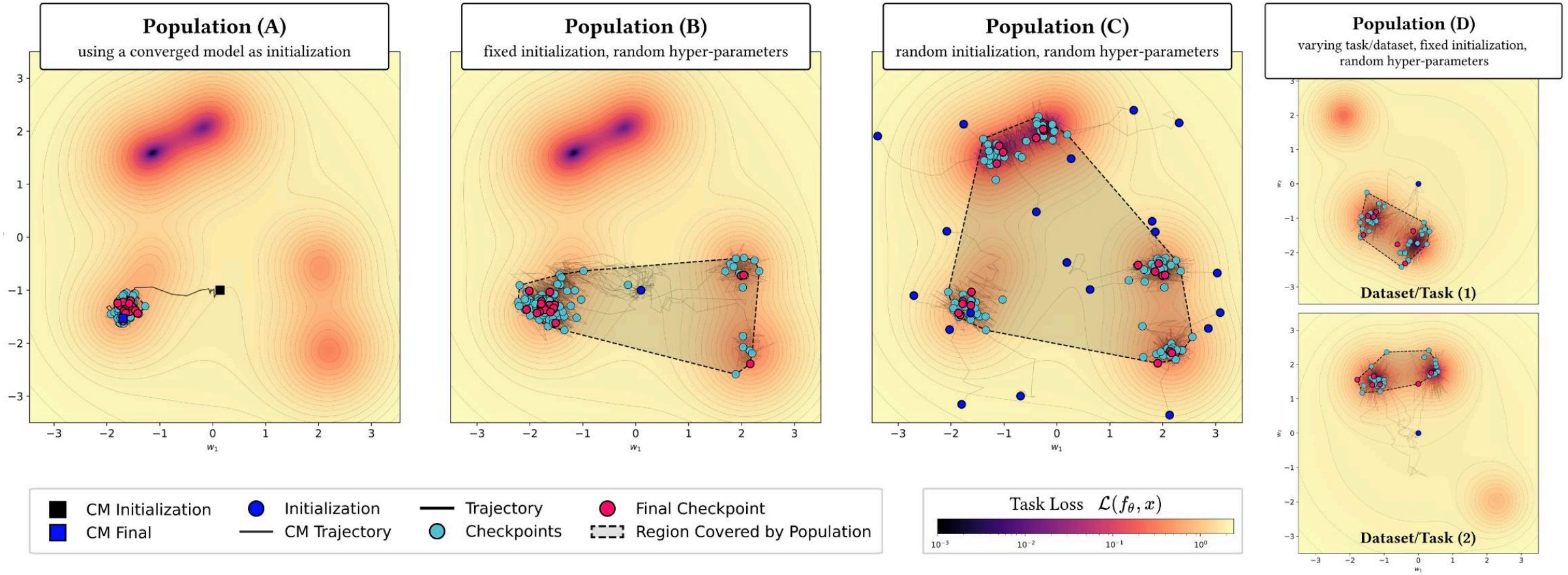
Hardware level non-determinism

Random Augmentation

Even when fixing the other factors, randomness during training can lead to different optimization trajectory.

Slides from Tobias Ettling

Model Population View



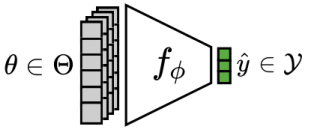
Weight Space Learning needs to unify benchmarking! A taxonomy of evaluation practices
T. Ettling, D. Falk, A. Asefaw, L. Meynent, K. Tzevelekakis, Z. Wang, K. Wang, B. Knyazev, H. Maron, D. Borth
(in submission)

Methodological View

Type	Methods	Backbone	Dis	Gen	$\mathcal{A}$	$\max(\lvert\theta_i\rvert)$	$\lvert\theta_i\rvert$
Weight Statistics	StatNN-1 [120]	Stats ∘ Light-GBM	✖			$7.4m$ ResNet	varying
	StatNN-2 [30]	Stats ∘ SVM & CNN	✖			$390k$ CNN	varying
	LinDet [134]	Stats & CNN ∘ Trf	✖			$86m$ ViT-B	fixed
	MoTHer [56]	Stats ∘ ~~MDST~~	✖			$86m$ ViT-B	fixed
	PhyDet [135]	Stats & CNN ∘ Trf	✖			$67m$ Lama3-PH	fixed
Transformer Autoencoder	HypRep [112]	Trf AE	✖			$2.5k$ CNN	fixed
	GenHypRep [111]	Trf AE [112]		✖		$2.5k$ CNN	fixed
	SANE [113]	Trf AE [111]	✖	✖		$44m$ ResNet101	varying
	SANE-B [80]	Trf AE [113]		✖		$11k$ CNN	fixed
	SANE-HF [34]	Trf AE [113]	✖	✖		$125m$ GPT-2	varying
Costume Equivariant Model Design	DWSNet [86]	DWS-Blocks	✖	✖		$209k$ MLP	fixed
	NFN [140]	NF-Blocks [86]	✖	✖		800^*k CNN	fixed
	NFT [141]	NF-Attn-Blocks	✖	✖		$5k$ CNN	fixed
	UNF [139]	NF-Blocks [140]	✖	✖		330^*k Transformer	fixed
	M-NFN [119]	MNF-Blocks [139]	✖	✖		$5k$ CNN	fixed
	GL-net [94]	GL-Blocks	✖			30^*m LoRA	fixed
Graph-based Models	GMN [73]	GNN & Graph Trf	✖	✖		$88k$ ViT-mini	varying
	NG-GNN [66]	GNN (PNA/FiLM)	✖	✖		$205k$ CNN	varying
	NG-T [66]	Graph Trf	✖	✖		$205k$ CNN	varying
	NiNo [65]	GNN [66]		✖		$111m$ Lama3-B	varying
	ScaleGMN [60]	GNN [73]	✖	✖		$5k$ CNN	fixed
Diffusion-based Model	G.pt [90]	Trf-CDM		✖		$8k$ MLP	fixed
	HyperDiff [31]	Trf-DM [90]		✖		$36k$ INR	fixed
	P-diff [127]	VAE ∘ LDM		✖		$80k$ ViT-mini	fixed
	D2NWG [116]	VAE ∘ CLDM		✖		$164m$ GPT2-small	varying
	COND P-diff [58]	DAE ∘ CLDM		✖		$1.1m$ LoRA	fixed
	GNN-diff [74]	VAE ∘ CLDM [127]		✖		$300k$ ChebNet	fixed
	RPG-diff [129]	Mamba ∘ DM		✖		$198m$ ConvNeXt-L	fixed
	ORAL [62]	Mamba ∘ CDM [129]		✖		50^*m LoRA	fixed
Other Methods for Weight Space Learning	RPG [126]	~~Training Method~~		✖		$21m$ ResNet34	fixed
	ModelZoo [114]	Linear Encoder	✖			$11m$ ResNet18	fixed
	NeRN [4]	Linear Encoder		✖		$11m$ ResNet18	fixed
	inr2vec [24]	Linear Encoder	✖			$790k$ INR	fixed
	ProbeGen [59]	Linear Encoder	✖			$205k$ CNN	fixed
	ProbePred [48]	LSTM Encoder	✖			$27k$ LSTM	fixed
	NeuMeta [133]	INR Decoder		✖		$25m$ ResNet50	varying
	SNE [2]	Set-Based Encoder	✖			$5k$ CNN	varying
	FLoWN [106]	FM ∘ [116, 66, 94]		✖		$11k$ CNN	fixed

MLP ●, INR ●, DeepSet ●, CNN ●, ResNet ●, ConvNeXt ●, Trf ●, ViT ●, RNN ●, GNN ●, LoRA ●.

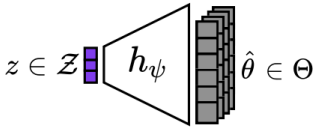
Encoder Only



Learn informative feature representations for Discriminative Downstream Tasks.

- Manual Feature Extraction (Shallow Learner)
- Convolution-based Encoder (1D or 2D Kernel)
- Linear-based Encoder (Parameter-/Activation-based)
- LSTM-based Encoder (Activation-based)
- Set-based Encoder (Set-to-Set and Set-to-Vector)

Decoder Only



Learn a conditional autoregressive model of the parameter distribution.

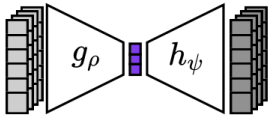
INR-Decoder:

Structural Metadata → INR Model → Parameters

RNN-Decoder:

Position & Model ID → RNN Model → Parameters

Autoencoder Models

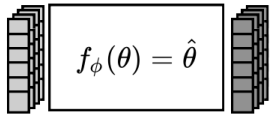


Learn a low-dimensional latent representation that preserves parameter information.

- Transformer Autoencoder
- Variational Autoencoder
- Denoising Autoencoder

→ Generative & Discriminative Tasks / Generative Tasks Only

Equivariant/GNN Models



Learn equivariant/invariant functions w.r.t. known parameter symmetries.

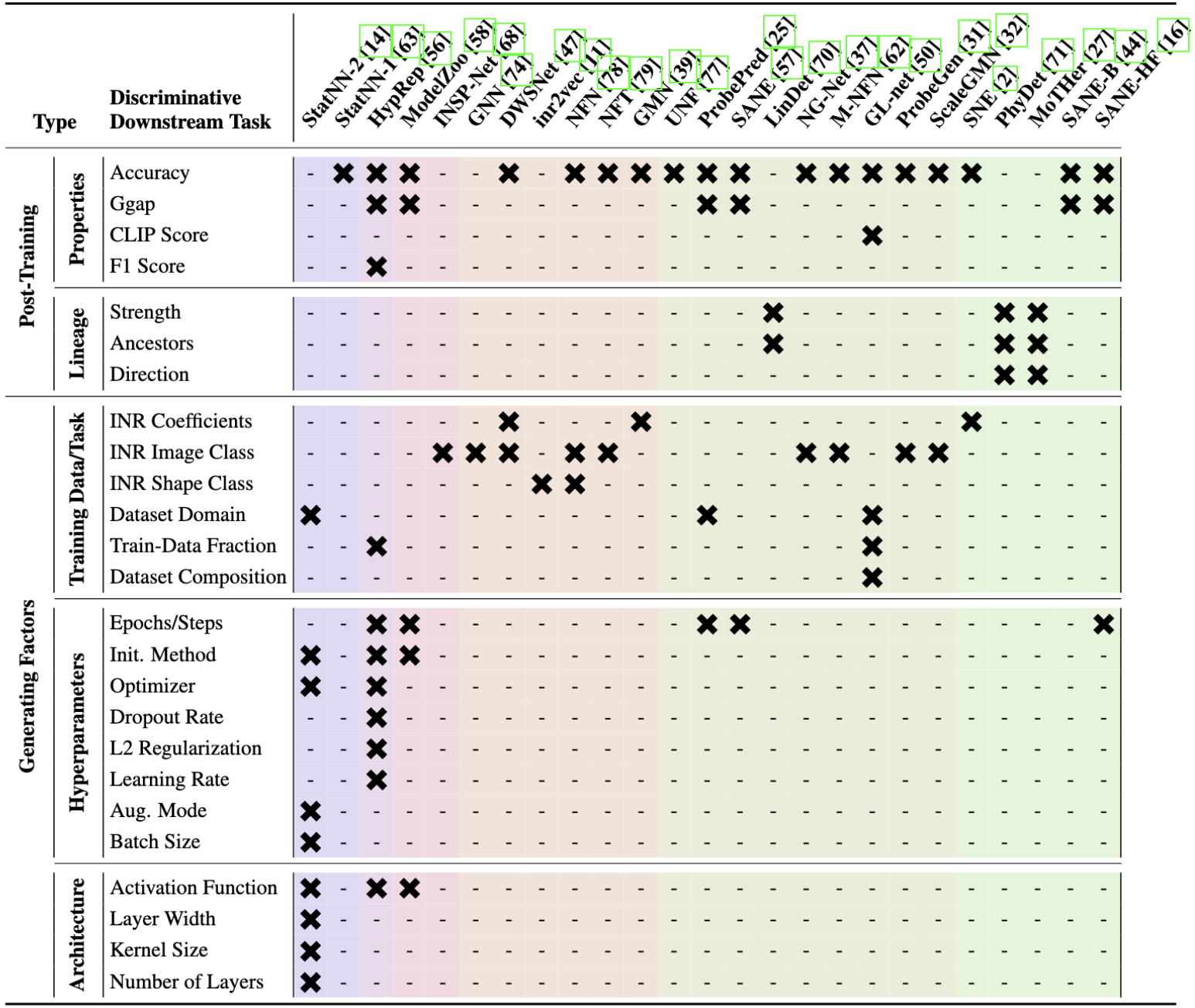
- Graph Neural Networks (Graph Structures)
- Custom Equivariant Models (Raw Parameter Tensors)

Parameters → Equivariant → Generative / Invariant → Discriminative

Slides from Tobias Ettling

Evaluation View:

Discriminative DST



2020 , 2021 , 2022 , 2023 , 2024 , 2025

Weight Space Learning needs to unify benchmarking!
A taxonomy of evaluation practices
T. Ettling, D. Falk, A. Asefaw, L. Meynent, K. Tzevelekakis, Z. Wang, K. Wang, B. Knyazev, H. Maron, D. Borth (in submission)

Evaluation View:

Generative DST

Gen. DST Grouping		Task Domain	Model Architecture	GenHypRep [55]	G.pt [49]	DWSNet [47]	NFN [78]	GNN [74]	HyperDiff [15]	NFT [79]	GMN [39]	UNF [77]	p-diff [64]	D2NWG [60]	GNN-diff [40]	SANE [57]	NG-Net [37]	COND p-diff [30]	M-NFN [62]	NeuMeta [69]	ScaleGMN [32]	RPG-diff [65]	NiNo [36]	FLoWN [53]	ORAL [33]	SANE-B [44]	SANE-HF [16]	GradMN [19]
Generating Full Model Weights	Weight Synthesis	Img. CLS	CNN	3	5	-	-	-	-	-	-	-	3	11	-	11	-	-	-	-	-	11	-	11	-	11	-	-
		Img. CLS	MLP	-	8	-	-	-	-	-	-	-	67	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		RL Cartpole	MLP	-	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		SF 3D/4D	INR	-	-	-	-	-	36	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		Img. CLS	RN	-	-	-	-	-	-	-	-	-	25	11	-	44	-	-	-	70	-	25	-	-	-	-	60	-
		Img. CLS	CNeXt	-	-	-	-	-	-	-	-	-	75	-	-	-	-	-	-	-	-	20	-	-	-	-	89	-
		Img. CLS	ViT	-	-	-	-	-	-	-	-	-	80	-	-	-	-	-	-	-	-	86	-	-	-	-	86	-
		Img. CLS	MobileNet	-	-	-	-	-	-	-	-	-	-	3	-	-	-	-	-	-	-	-	-	-	-	-	3	-
		Text Gen.	Tx	-	-	-	-	-	-	-	-	-	-	16	-	-	-	-	-	-	-	-	-	-	-	-	13	-
		Node CLS	GNN	-	-	-	-	-	-	-	-	-	-	-	30	-	-	-	-	-	-	-	-	-	-	-	-	-
		Img. Gen.	VAE	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	50	-	-	-	-	-	-	-	-
		Img. Seg.	RN	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	14	-	-	-	-	-	-	-	-
		Img. Seg.	ViT	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	18	-	-	-	-	-	-
		Img. Det./Seg.	ViT	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	-	-	-	-	86	-
	Edit	Img. CLS	MLP	-	-	14	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		SF 2D	INR	-	-	-	2	2	-	2	2	-	-	-	-	-	2	-	2	-	2	-	-	-	-	-	-	2
	Mask	Img. CLS	MLP	-	-	-	25	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		Img. CLS	CNN	-	-	-	13	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
	Optimize	Img. CLS	MLP	-	-	-	-	-	-	-	-	3	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	54
		Img. CLS	CNN	-	-	-	-	-	-	-	-	5	-	-	-	-	15	-	-	-	-	-	63	-	-	-	-	-
		Text Gen.	RNN	-	-	-	-	-	-	-	-	15	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		Text Gen.	Tx	-	-	-	-	-	-	-	-	25	-	-	-	-	-	-	-	-	-	-	11	-	-	-	-	25
Generating Partial Model Weights	Weight Synthesis	Img. CLS	RN (BN)	-	-	-	-	-	-	-	-	2	-	-	-	-	-	-	-	-	-	-	-	2	-	-	-	-
		Img. CLS	ViT (BN)	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-	-	3	-	-	-	-
		Img. CLS	CNeXt (BN)	-	-	-	-	-	-	-	-	3	-	-	-	-	-	-	-	-	-	-	-	3	-	-	-	-
		Img. Det.	CNN (sub)	-	-	-	-	-	-	-	-	5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		Img. Seg.	CNN (BN)	-	-	-	-	-	-	-	-	4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		Img. Gen.	RN (sub)	-	-	-	-	-	-	-	-	4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		Img. CLS	RN (CH)	-	-	-	-	-	-	-	-	-	8	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		Img. CLS	Tx (CH)	-	-	-	-	-	-	-	-	-	13	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		Text CLS	Tx (LoRA)	-	-	-	-	-	-	-	-	-	60	-	-	-	1	-	-	-	-	-	-	-	10	-	-	-
		Text Gen.	Tx (top 25%)	-	-	-	-	-	-	-	-	-	87	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
		Img. Gen.	DiT (LoRA)	-	-	-	-	-	-	-	-	-	-	-	-	-	50	-	-	-	-	-	-	-	4	-	-	-
		Text Gen.	Tx (DoRA)	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	11	-	-	-	-	-	-
		IT-Text	ViT-Tx (LoRA)	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	10	-	-	-

Image (**Img.**); Classification (**CLS**); Signal Fit (**SF**); Generation (**Gen.**); Segmentation (**Seg.**); Detection (**Det.**); Image-Text-to-Text (**IT-Text**); Transformer (**Tx**); ResNet (**RN**); Bach Normalization Weights (**BN**); Weight Subset (**sub**); Classification Head (**CH**)

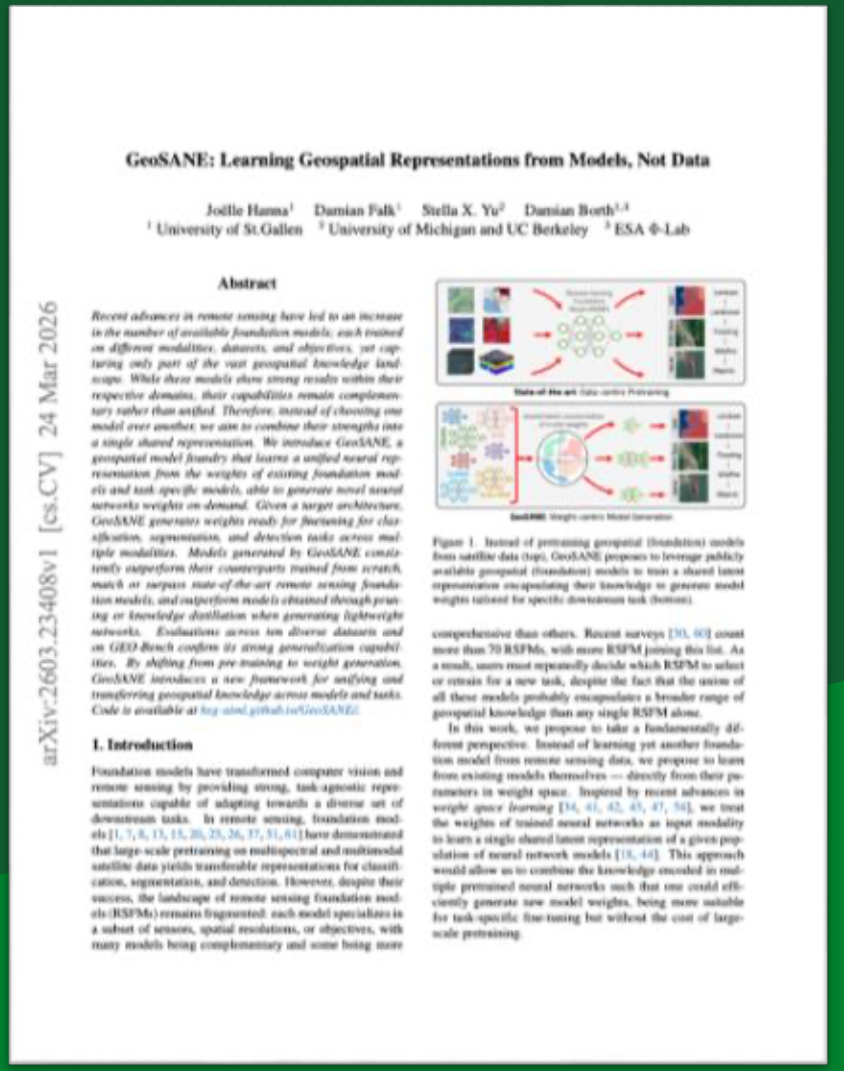
2022, 2023, 2024, 2025.

K $= 10^3$, **10K** $= 10^4$, **M** $= 10^6$, **10M** $= 10^7$.

Weight Space Learning needs to unify benchmarking!
A taxonomy of evaluation practices
T. Ettling, D. Falk, A. Asefaw, L. Meynent, K. Tzevelekakis, Z. Wang, K. Wang, B. Knyazev, H. Maron, D. Borth
(in submission)

GeoSANE

J. Hanna, D.Falk, Stella X Yu,, D Borth
GeoSANE: Learning Geospatial Representations from Models, Not Data
IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2026



GeoSANE: Learning Geospatial Representations from Models, Not Data

Joëlle Hanna[1] Damian Falk[1] Stella X. Yu[2] Damian Borth[1,3]
[1] University of St.Gallen [2] University of Michigan and UC Berkeley [3] ESA Φ-Lab

arXiv:2603.23408v1 [cs.CV] 24 Mar 2026

Abstract

Recent advances in remote sensing have led to an increase in the number of available foundation models, each trained on different modalities, datasets, and objectives, yet capturing only part of the vast geospatial knowledge landscape. While these models show strong results within their respective domains, their capabilities remain complementary rather than unified. Therefore, instead of choosing one model over another, we aim to combine their strengths into a single shared representation. We introduce GeoSANE, a geospatial model foundry that learns a unified neural representation from the weights of existing foundation models and task-specific models, able to generate novel neural networks weights on demand. Given a target architecture, GeoSANE generates weights ready for finetuning for classification, segmentation, and detection tasks across multiple modalities. Models generated by GeoSANE consistently outperform their counterparts trained from scratch, match or surpass state-of-the-art remote sensing foundation models, and outperform models obtained through pruning or knowledge distillation when generating lightweight networks. Evaluations across ten diverse datasets and on GEO-Bench confirm its strong generalization capabilities. By shifting from pre-training to weight generation, GeoSANE introduces a new framework for unifying and transferring geospatial knowledge across models and tasks. Code is available at https://github.com/GeoSANE/.

1. Introduction

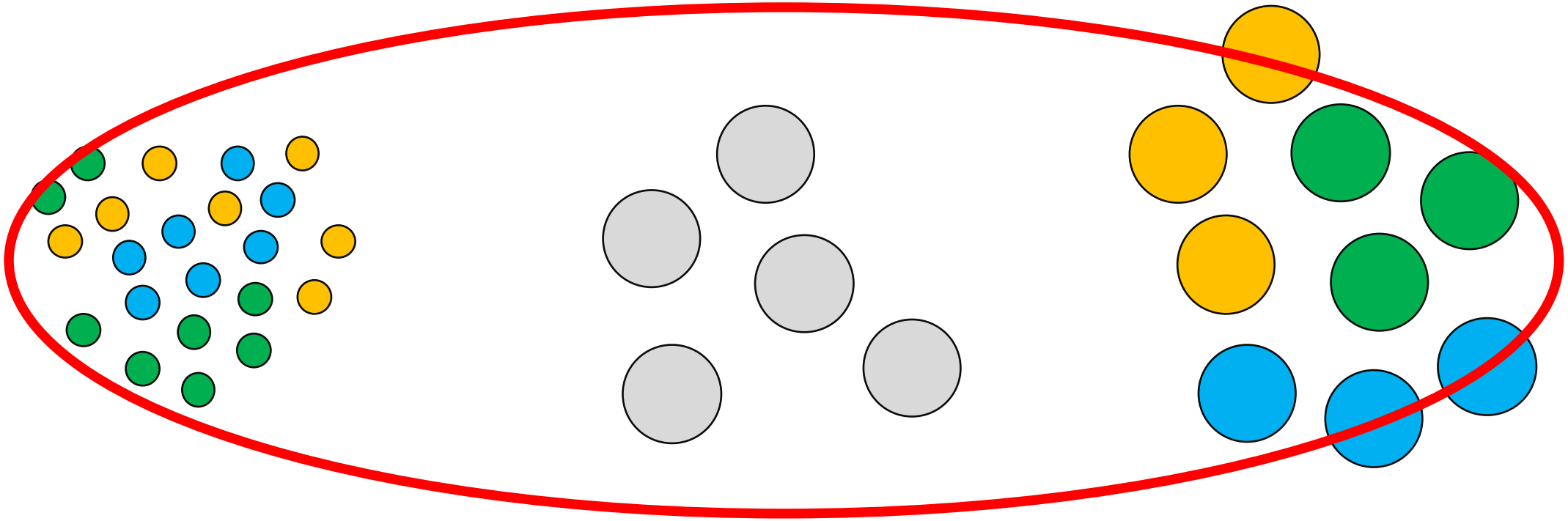
Foundation models have transformed computer vision and remote sensing by providing strong, task-agnostic representations capable of adapting towards a diverse set of downstream tasks. In remote sensing, foundation models [1, 7, 8, 13, 15, 20, 25, 26, 37, 51, 61] have demonstrated that large-scale pretraining on multispectral and multimodal satellite data yields transferable representations for classification, segmentation, and detection. However, despite their success, the landscape of remote sensing foundation models (RSFMs) remains fragmented: each model specializes in a subset of sensors, spatial resolutions, or objectives, with many models being complementary and some being more comprehensive than others. Recent surveys [30, 60] count more than 70 RSFMs, with more RSFM joining this list. As a result, users must repeatedly decide which RSFM to select or retrain for a new task, despite the fact that the union of all these models probably encapsulates a broader range of geospatial knowledge than any single RSFM alone.

In this work, we propose to take a fundamentally different perspective. Instead of learning yet another foundation model from remote sensing data, we propose to learn from existing models themselves — directly from their parameters in weight space. Inspired by recent advances in *weight space learning* [34, 41, 42, 45, 47, 58], we treat the weights of trained neural networks as input modality to learn a single shared latent representation of a given population of neural network models [18, 44]. This approach would allow us to combine the knowledge encoded in multiple pretrained neural networks such that one could efficiently generate new model weights, being more suitable for task-specific fine-tuning but without the cost of large-scale pretraining.

Figure 1. Instead of pretraining geospatial (foundation) models from satellite data (top), GeoSANE proposes to leverage publicly available geospatial (foundation) models to train a shared latent representation encapsulating their knowledge to generate model weights tailored for specific downstream task (bottom).

Geo-spatial Foundation Models

Can we learn from all this pretrained models and "**extract**" their knowledge to sample geospatial models on demand?

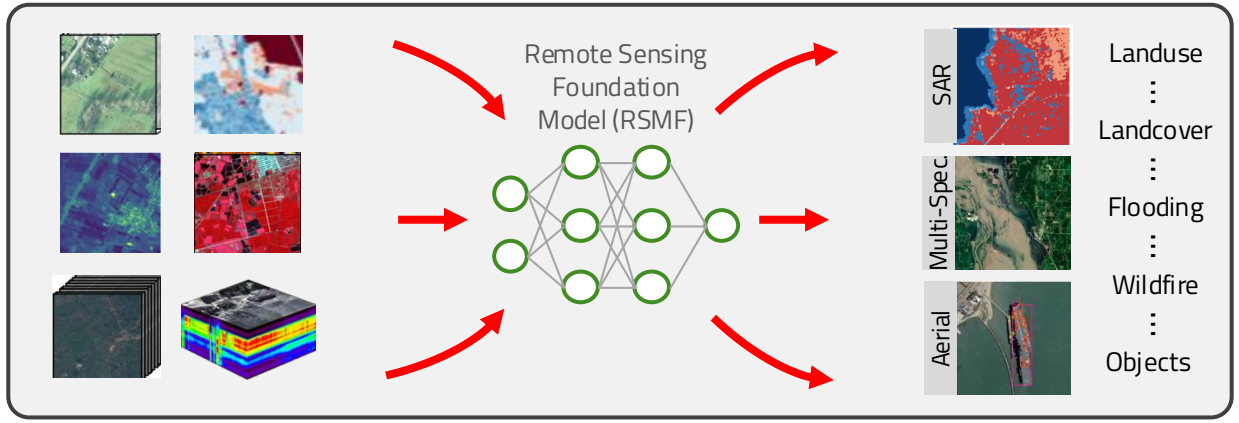


Model Zoo of **specialized** pre-trained RS/EO models

Model Zoo of RS/EO **foundation models**

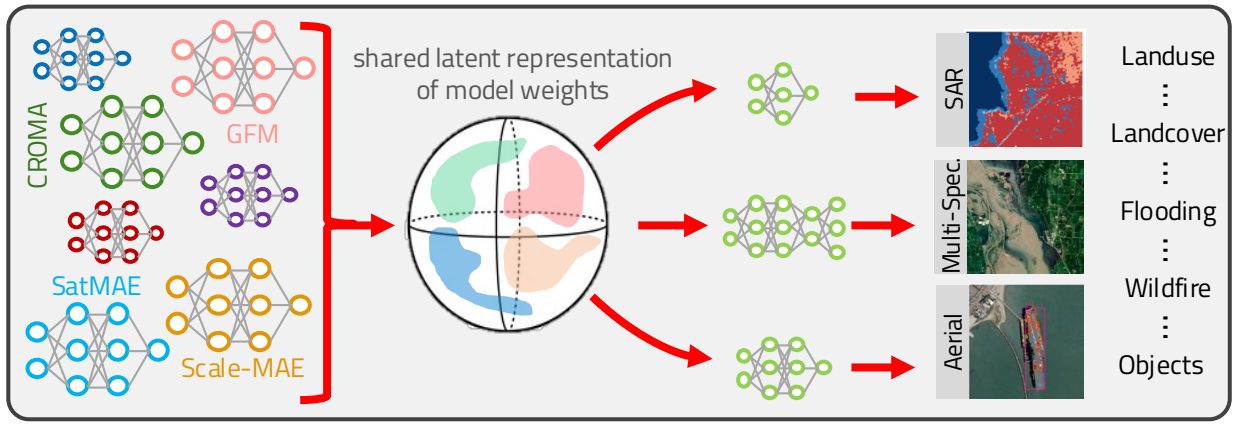
Model Zoo of **downstream-task specialized** RS / EO foundation models

Idea: Learning from Models, not Data



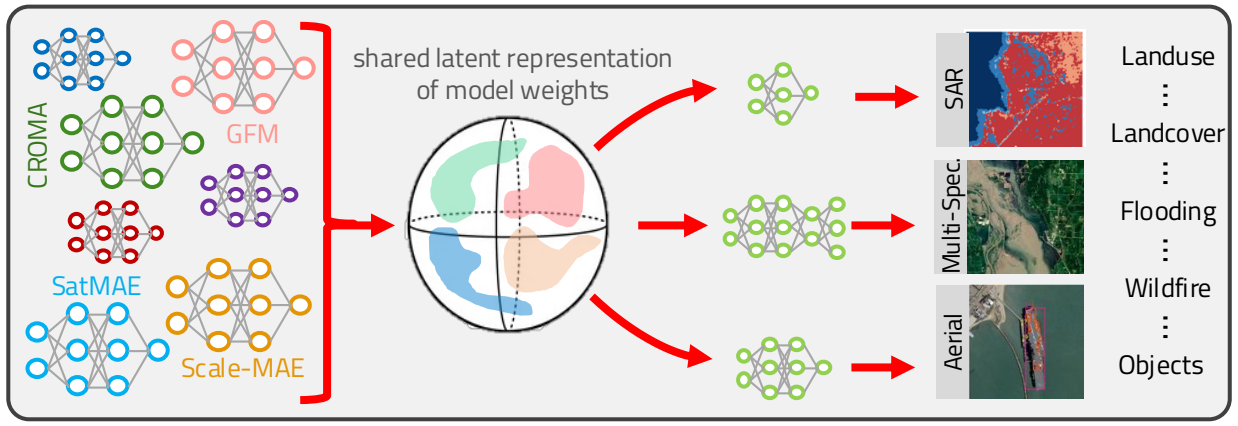
State-of-the-art: Data-centric Pretraining

Idea: Learning from Models, not Data

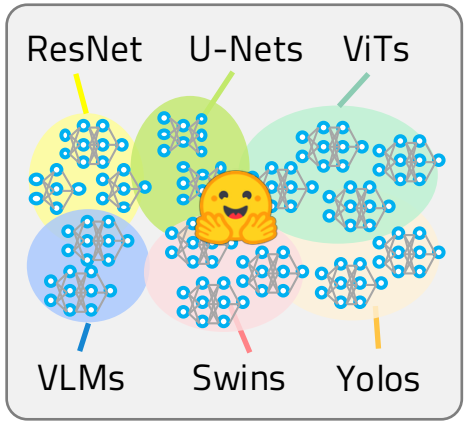


GeoSANE: Weight-centric Model Generation

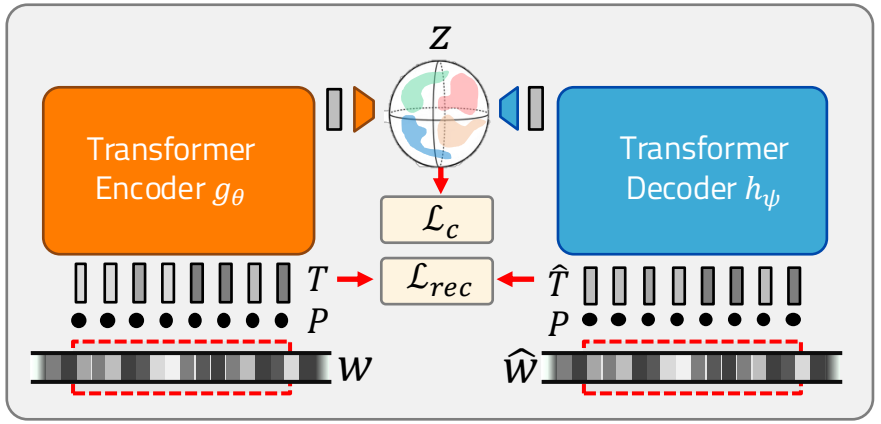
Idea: Learning from Models, not Data



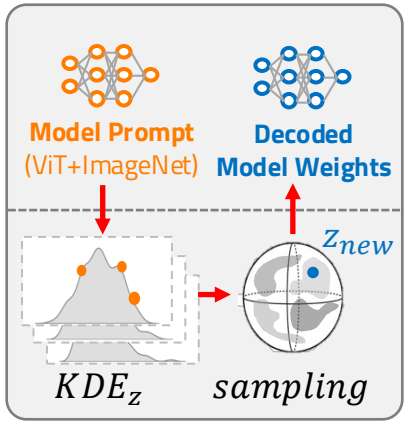
GeoSANE: Weight-centric Model Generation



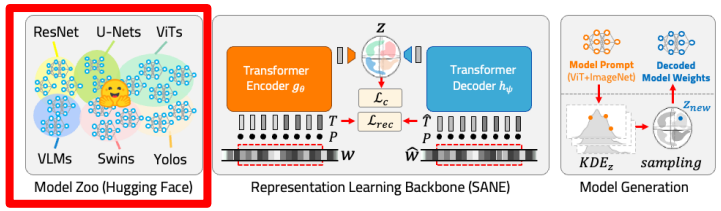
Model Zoo (Hugging Face)



Representation Learning Backbone (SANE)



Model Generation



Hugging Face Model Zoo

INTERNET ARCHIVE **Gen AI Archive**



NZZ

INTERVIEW

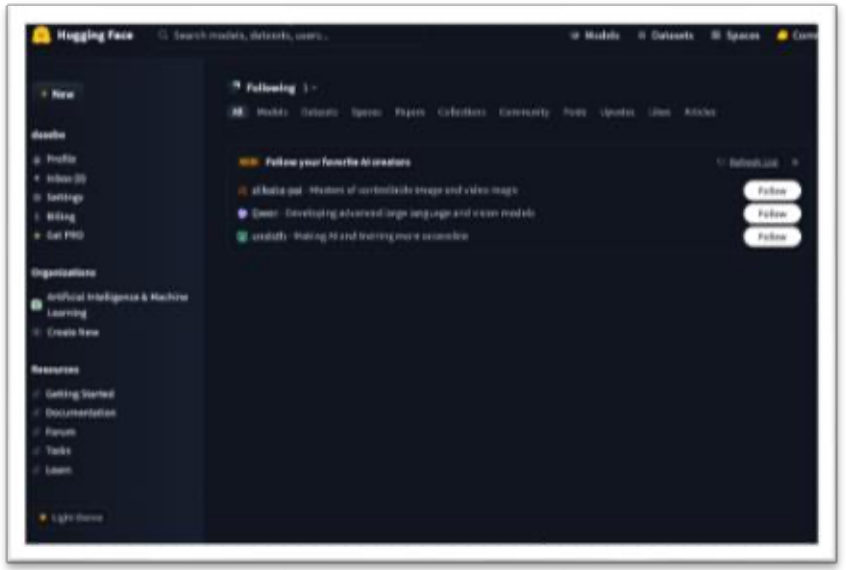
«In hundert Jahren werden Historiker die KI interviewen, um Epochen zu erforschen»

Mönche, Modelle, Metadaten – der Computerwissenschafter Damian Borth spricht im Interview darüber, warum es für die Menschheit so wichtig ist, dass die künstliche Intelligenz in das Stiftsarchiv St. Gallen einzieht.

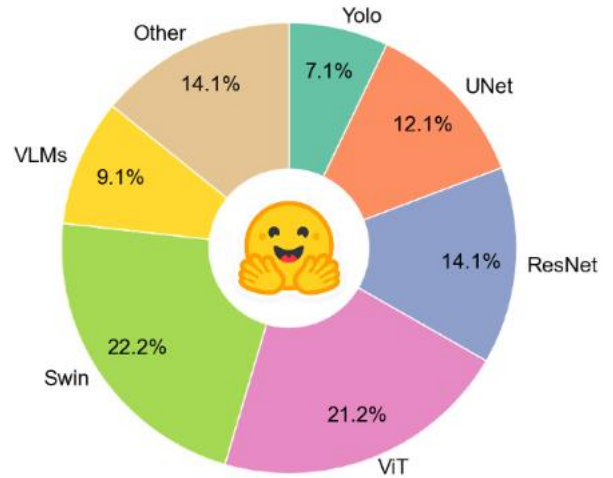
Barnaby Skinner

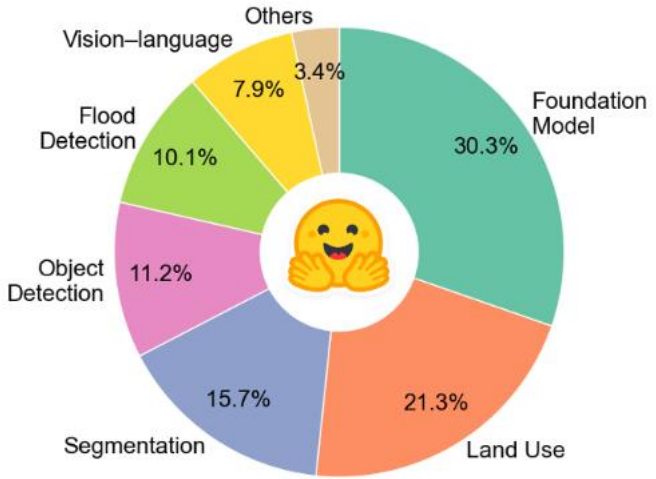
NZZ, Saturday, June 13th 2026

=

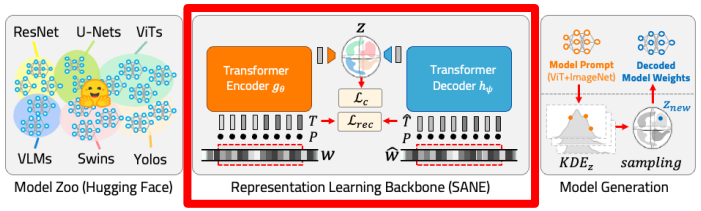


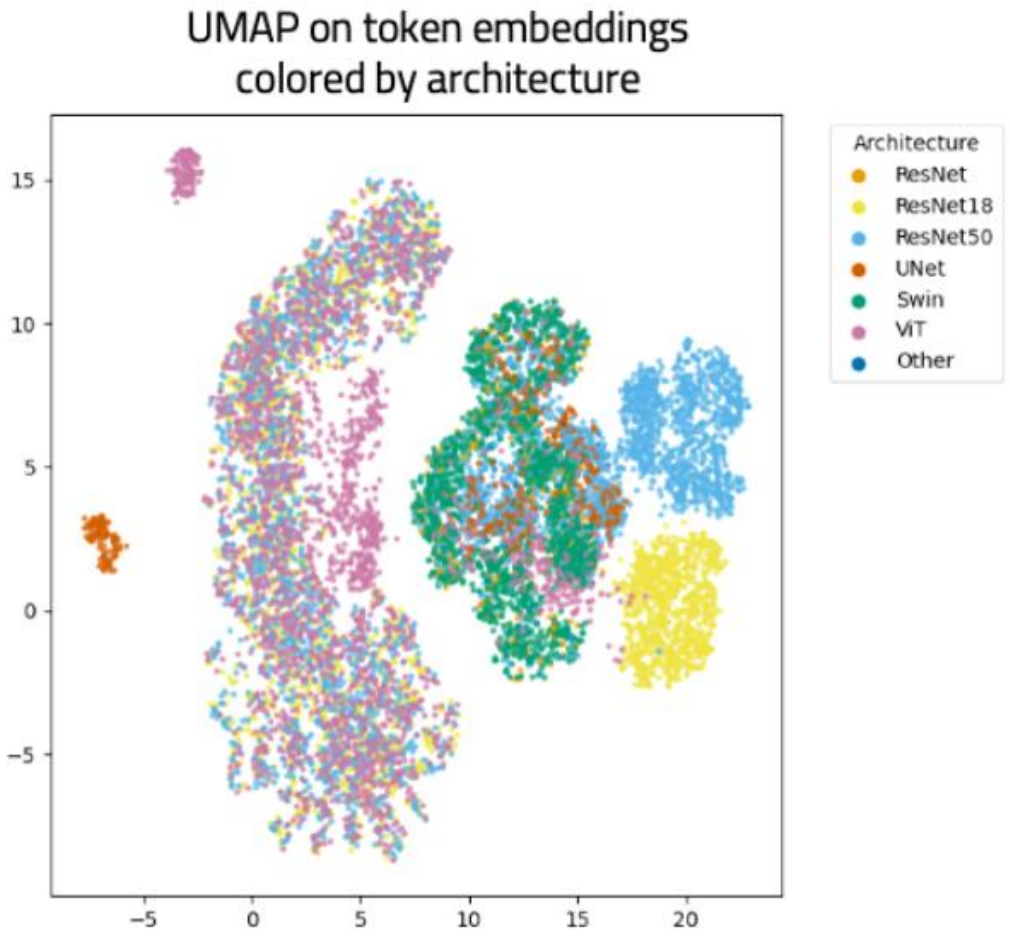
Model Zoo:
~**100** RS/EO models
~**38 billion** parameters

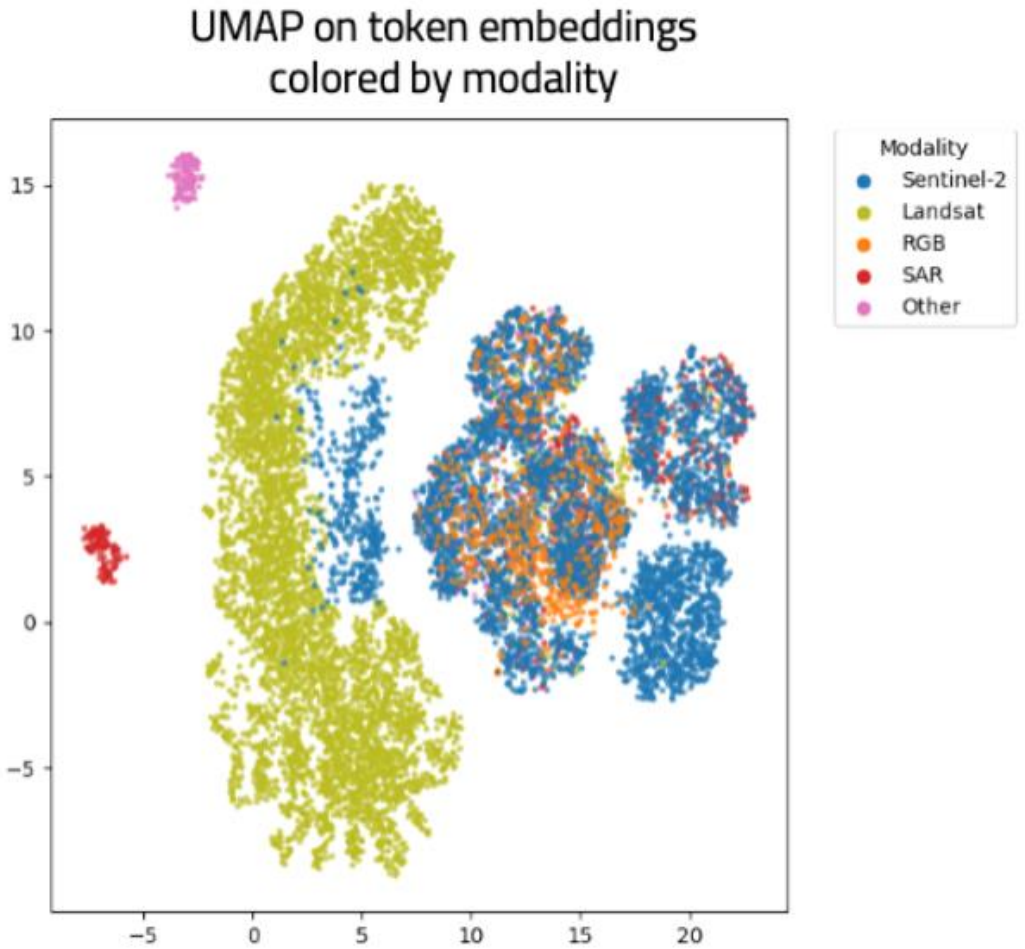


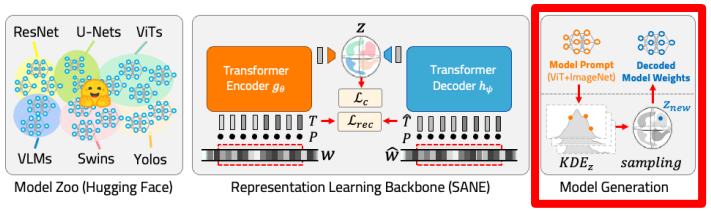


Learned Shared Latent Space









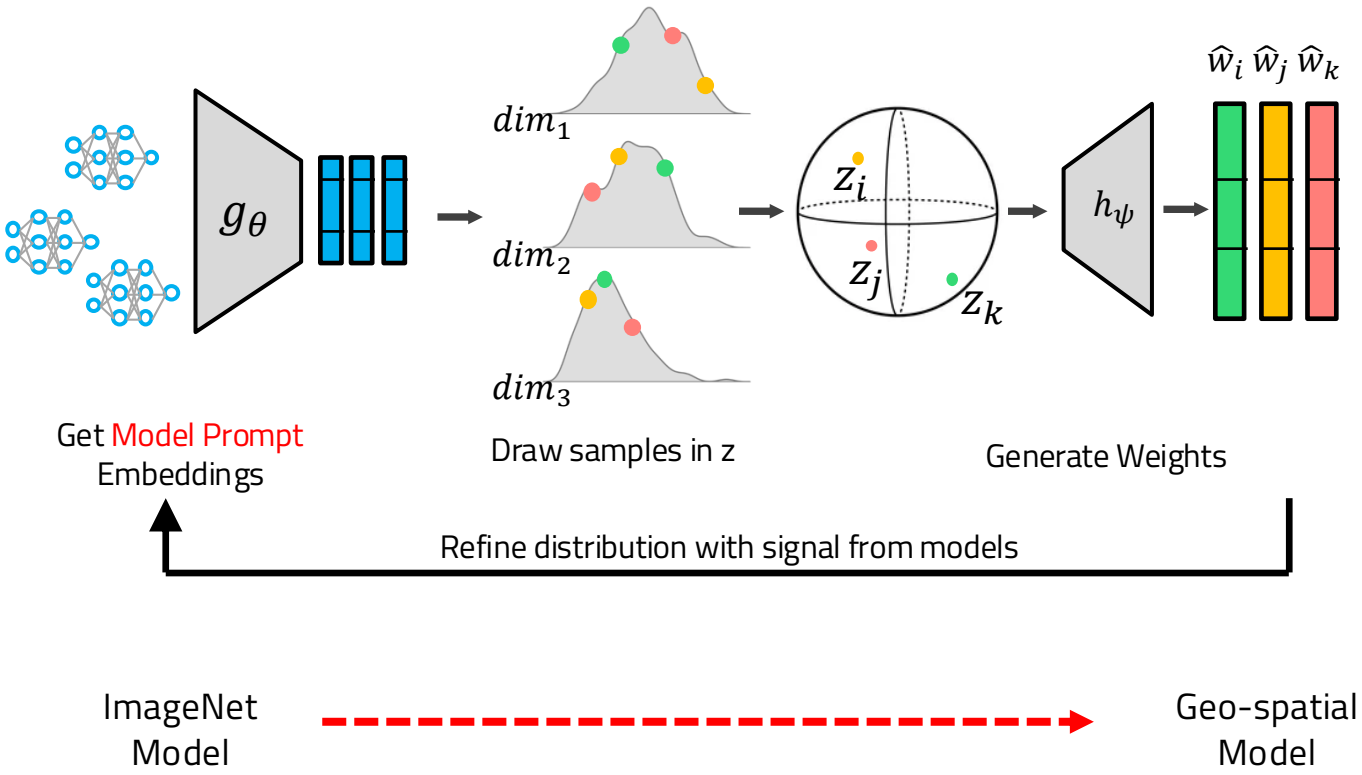
Model Generation

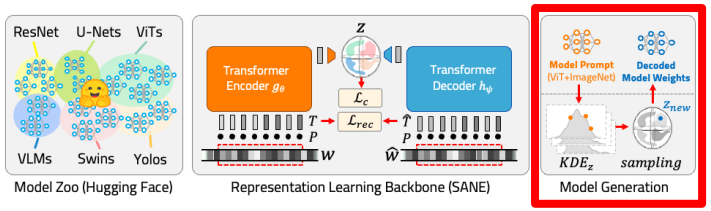
Problem:

- sampling requires a large number of high-quality models to identify the distribution in latent space

Approach:

- start with an ImageNet pretrained model
- sample broadly and evaluate generated models
- Refine n-times

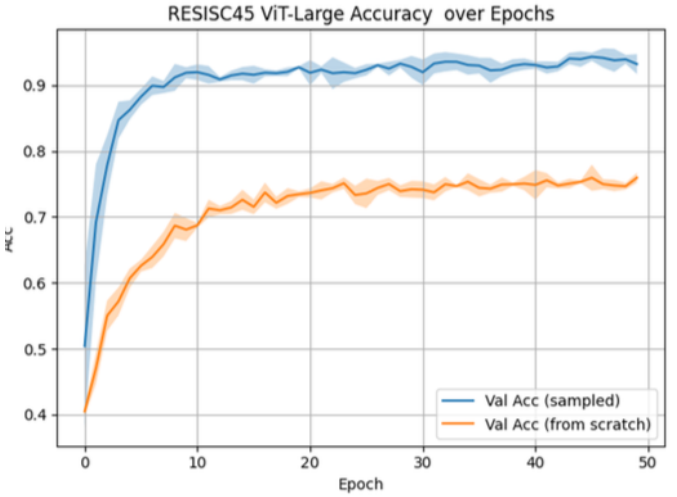


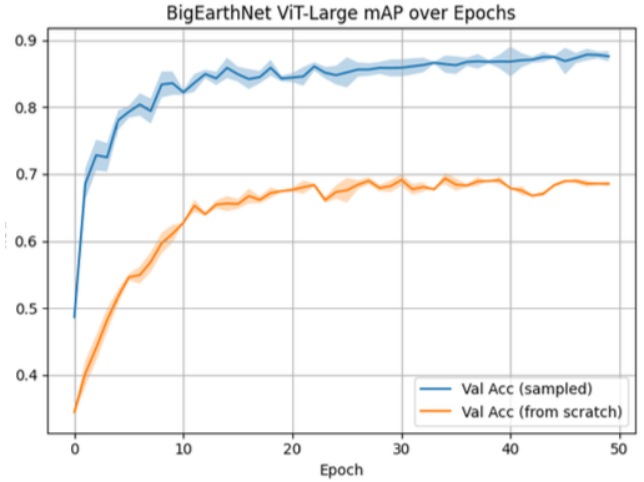


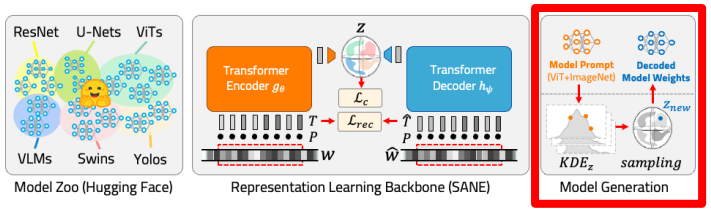
Results – Training from Scratch

Q1: Does GeoSANE provide better initialization than training from scratch?

Dataset	Backbone	from scratch	GeoSANE	Δ
EuroSAT [17]	ViT-L	95.0	**99.1**	+4.1
RESISC-45 [5]	ViT-L	78.0	**96.5**	+18.5
fMoW [6]	ViT-L	34.1	**58.9**	+24.8
Sen12Flood [30]	ViT-L	80.4	**85.2**	+4.8
Cal. Wildfires [4]	ViT-L	88.7	**94.9**	+6.2
BigEarthNet [40]	ViT-L	68.3	**88.7**	+20.4
DFC2020 [34]	Swin-B	46.8	**54.3**	+7.5
Spacenet1 [10]	Swin-B	72.2	**78.2**	+6.0
Sen1Floods11 [2]	Swin-B	81.0	**89.6**	+8.6
DIOR [22]	Swin-B	67.5	**79.0**	+11.5







Results – Comparison to other RSFM

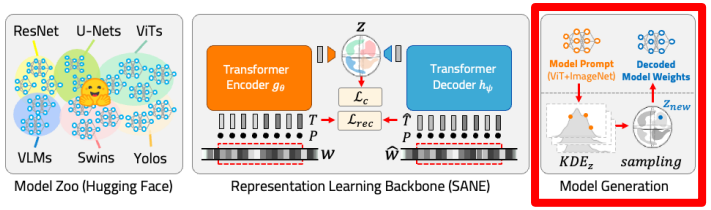
Q2: How do the generated models compare to existing remote sensing foundation models?

Reported Performance

Model	Backbone	Single-label					Multi-label	Segmentation			Object Det.
		EuroSAT	RESISC45	fMoW	Sen12Flood	Wildfires	BigEarthNet	DFC2020	SpaceNet	Sen1Floods11	DIOR
SatMAE [7]	ViT-L	98.9	94.8	58.2	80.3	88.6	86.2	44.1	78.1	–	70.9
Scale-MAE [31]	ViT-L	99.1	95.7	–	82.6	90.8	87.9	–	**78.9**	74.1	73.8
RingMo [41]	Swin-B	–	95.7	–	–	–	–	–	–	–	75.9
CROMA [12]	ViT-L	**99.4**	–	**59.0**	83.4	93.3	88.3	49.8	–	**90.9**	–
GFM [26]	Swin-B	–	–	–	77.9	91.9	86.3	–	–	72.6	–
SkySense [14]	ViT-L	–	–	–	–	–	88.6	–	–	–	78.7
DOFA [50]	ViT-L	–	**97.3**	–	–	–	–	–	–	89.4	–
GeoSANE	ViT-L or Swin-B	99.1±0.2	96.5 ±0.1	58.9±0.1	**85.2**±0.3	**94.9**±0.2	**88.7**±0.1	**54.3**±0.3	78.2±0.3	89.6±0.1	**79.0**±0.2
Δ		-0.3	-0.8	-0.1	+1.8	+1.6	+0.1	+4.5	-0.7	-1.3	+0.3

Geo-Bench

Method	Backbone	m-eurosat	m-bigearthnet	m-so2sat	m-brick-kiln
SatMAE [7]	ViT-L	96.6	68.3	57.2	98.4
CROMA [12]	ViT-L	96.6	71.9	60.6	98.7
DOFA [50]	ViT-L	96.9	68.0	58.7	98.6
Prithvi-EO 2.0 [42]	ViT-L	96.5	69.0	54.6	98.6
AnySat [1]	ViT-B	95.9	70.3	51.8	98.6
Galileo [44]	ViT-B	97.7	70.7	63.3	98.7
TerraFM [8]	ViT-L	**98.6**	73.1	64.9	**99.0**
GeoSANE	ViT-L	97.7±0.1	**74.2**±0.3	**65.7**±0.2	98.6±0.2
Δ		-0.9	+1.1	+0.8	-0.4



Model Zoo (Hugging Face) | Representation Learning Backbone (SANE) | Model Generation

Results – Comparis

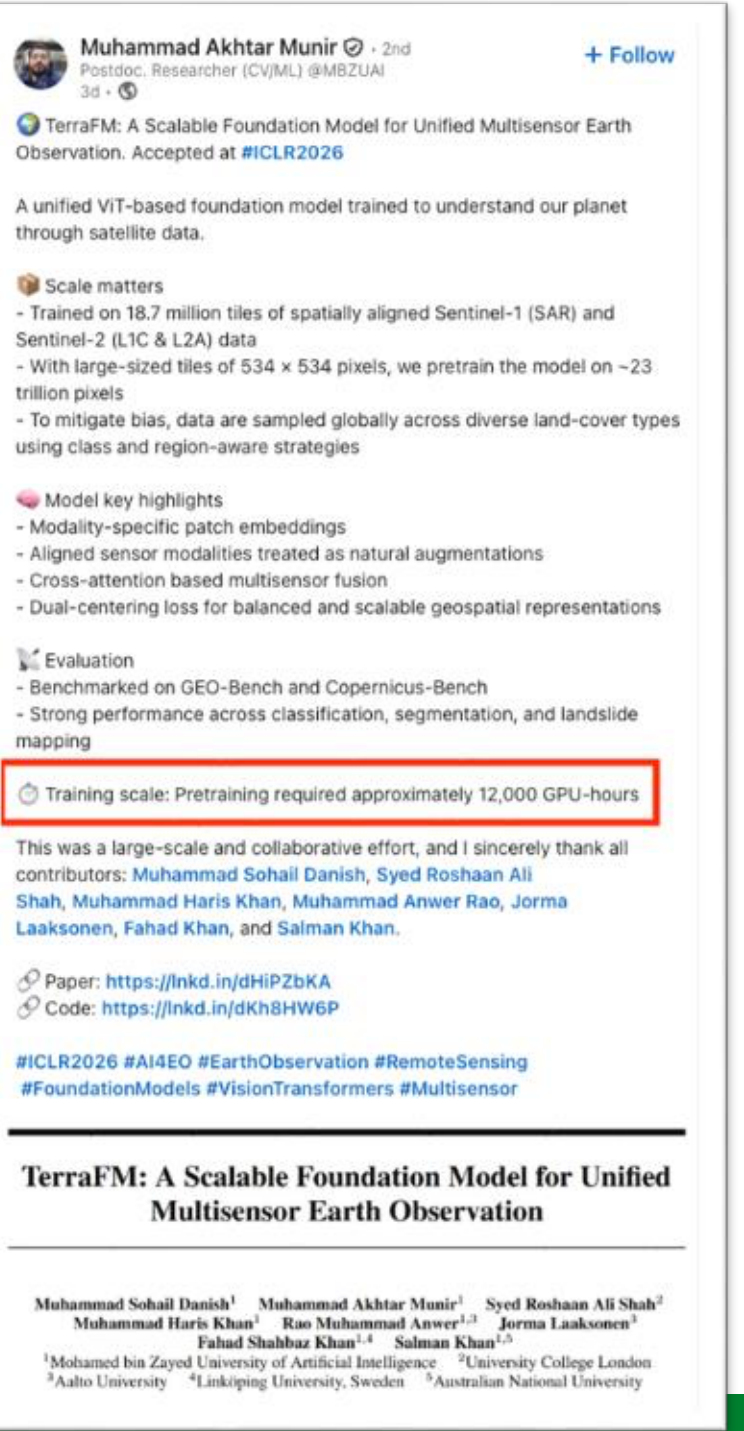
Q2: How do the generated models compare to exis

Reported Performance

Model	Backbone	Single-label					Multi-label	
		EuroSAT	RESISC45	fMoW	Sen12Flood	Wildfires	BigEarthNet	DFC2(
SatMAE [7]	ViT-L	98.9	94.8	58.2	80.3	88.6	86.2	44.1
Scale-MAE [31]	ViT-L	99.1	95.7	–	82.6	90.8	87.9	–
RingMo [41]	Swin-B	–	95.7	–	–	–	–	–
CROMA [12]	ViT-L	**99.4**	–	**59.0**	83.4	93.3	88.3	49.8
GFM [26]	Swin-B	–	–	–	77.9	91.9	86.3	–
SkySense [14]	ViT-L	–	–	–	–	–	88.6	–
DOFA [50]	ViT-L	–	**97.3**	–	–	–	–	–
GeoSANE	ViT-L or Swin-B	99.1±0.2	96.5 ±0.1	58.9±0.1	**85.2**±0.3	**94.9**±0.2	**88.7**±0.1	**54.3**±
Δ		-0.3	-0.8	-0.1	+1.8	+1.6	+0.1	+4.5

Geo-Bench

Method	Backbone	m-eurosat	m-bigearthnet	m-so2sat	m-brick-kiln
SatMAE [7]	ViT-L	96.6	68.3	57.2	98.4
CROMA [12]	ViT-L	96.6	71.9	60.6	98.7
DOFA [50]	ViT-L	96.9	68.0	58.7	98.6
Prithvi-EO 2.0 [42]	ViT-L	96.5	69.0	54.6	98.6
AnySat [1]	ViT-B	95.9	70.3	51.8	98.6
Galileo [44]	ViT-B	97.7	70.7	63.3	98.7
TerraFM [8]	ViT-L	**98.6**	73.1	64.9	**99.0**
GeoSANE	ViT-L	97.7±0.1	**74.2**±0.3	**65.7**±0.2	98.6±0.2
Δ		-0.9	+1.1	+0.8	-0.4



Muhammad Akhtar Munir · 2nd
Postdoc. Researcher (CV/ML) @MBZUAI
3d · + Follow

TerraFM: A Scalable Foundation Model for Unified Multisensor Earth Observation. Accepted at #ICLR2026

A unified ViT-based foundation model trained to understand our planet through satellite data.

Scale matters
- Trained on 18.7 million tiles of spatially aligned Sentinel-1 (SAR) and Sentinel-2 (L1C & L2A) data
- With large-sized tiles of 534 × 534 pixels, we pretrain the model on ~23 trillion pixels
- To mitigate bias, data are sampled globally across diverse land-cover types using class and region-aware strategies

Model key highlights
- Modality-specific patch embeddings
- Aligned sensor modalities treated as natural augmentations
- Cross-attention based multisensor fusion
- Dual-centering loss for balanced and scalable geospatial representations

Evaluation
- Benchmarked on GEO-Bench and Copernicus-Bench
- Strong performance across classification, segmentation, and landslide mapping

Training scale: Pretraining required approximately 12,000 GPU-hours

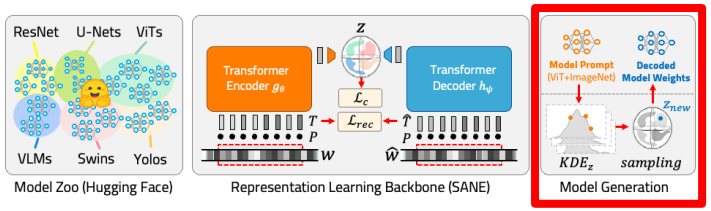
This was a large-scale and collaborative effort, and I sincerely thank all contributors: Muhammad Sohail Danish, Syed Roshaan Ali Shah, Muhammad Haris Khan, Muhammad Anwer Rao, Jorma Laaksonen, Fahad Khan, and Salman Khan.

Paper: https://lnkd.in/dHiPZbKA
Code: https://lnkd.in/dKh8HW6P

#ICLR2026 #AI4EO #EarthObservation #RemoteSensing #FoundationModels #VisionTransformers #Multisensor

TerraFM: A Scalable Foundation Model for Unified Multisensor Earth Observation

Muhammad Sohail Danish[1] Muhammad Akhtar Munir[1] Syed Roshaan Ali Shah[2]
Muhammad Haris Khan[1] Rao Muhammad Anwer[1,3] Jorma Laaksonen[3]
Fahad Shahbaz Khan[1,4] Salman Khan[1,5]
[1]Mohamed bin Zayed University of Artificial Intelligence [2]University College London
[3]Aalto University [4]Linköping University, Sweden [5]Australian National University

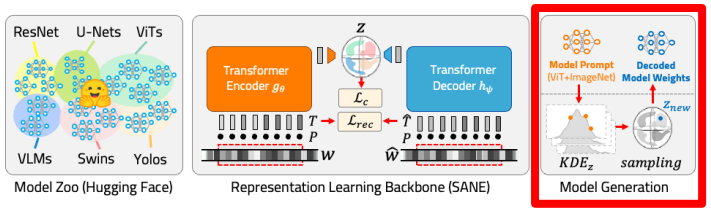


Results – Comparison to Distillation

Q4: Can GeoSANE generate strong lightweight models without explicit compression?

	Dataset	Magnitude Pruning (MP) Initial Models			Variational Dropout (VP) Initial Models			Knowledge Distillation (KD) Teachers			GeoSANE	Δ
		ViT-L (IN)	ScaleMAE	SatMAE	ViT-L (IN)	ScaleMAE	SatMAE	ViT-L (IN)	ScaleMAE	SatMAE		
11M Params	RESISC-45	88.1	87.2	84.9	81.1	81.8	80.6	90.2	90.3	91.1	**92.2**	+1.1
	EuroSAT	97.7	95.4	96.3	95.5	97.0	94.7	97.9	94.0	94.7	**98.7**	+0.8
	fMoW	33.7	36.2	35.4	25.2	32.3	27.1	38.6	43.1	41.9	**53.5**	+10.4
	BigEarthNet	45.6	59.1	62.1	38.2	44.9	49.8	65.7	67.3	66.5	**83.7**	+16.4
	Sen12Flood	77.2	79.0	76.5	75.4	75.3	77.8	79.3	82.1	80.8	**84.0**	+1.9
	Cal. Wildfires	82.9	82.7	84.6	79.1	79.8	81.3	85.2	87.6	88.0	**91.6**	+3.6
3.5M Params*	RESISC-45	64.9	65.6	63.6	61.2	62.0	61.0	67.2	67.5	68.1	**70.0**	+1.9
	EuroSAT	89.2	90.8	91.3	85.5	88.9	90.7	92.1	93.4	94.8	**96.2**	+1.4
	fMoW	15.2	16.7	20.5	16.5	17.8	17.4	23.2	**25.5**	22.1	17.7	-7.8
	BigEarthNet	42.5	41.8	43.9	32.8	34.6	37.1	55.1	60.7	58.5	**73.3**	+12.6
	Sen12Flood	60.1	62.7	62.8	55.9	58.4	59.5	62.1	62.8	64.7	**70.2**	+5.5
	Cal. Wildfires	68.0	69.6	69.9	62.9	63.7	66.2	72.9	74.1	75.4	**75.9**	+0.5

* For MP and VP, models were pruned to approx. 5M parameters instead of 3.5M, as lower sparsity caused model instability.



Results – Model Merging / Model Prompt

Model	Single-label		Multi-label
	EuroSAT	RESISC45	BigEarthNet
SatMAE [7]	98.9	94.8	86.2
Scale-MAE [37]	**99.1**	95.7	87.9
Merged Model	96.4	86.1	69.0
GeoSANE	**99.1**	**96.5**	**88.7**

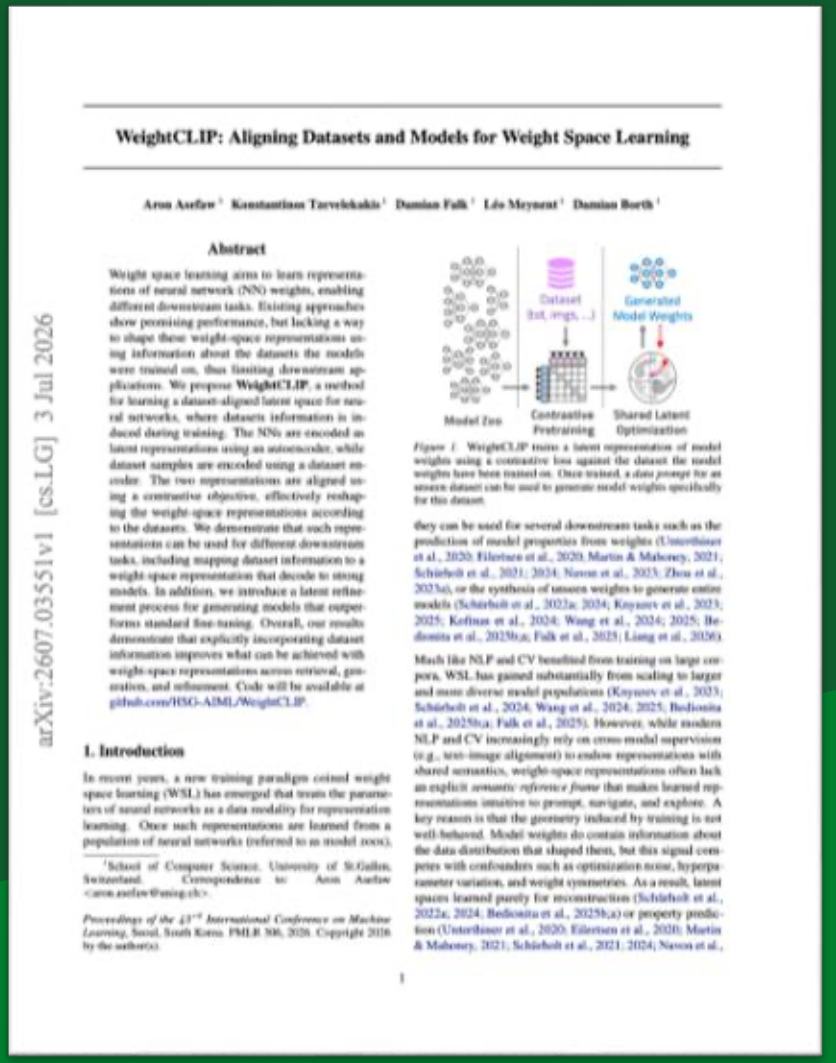
Dataset	Model Prompt	**GeoSANE**	Δ
EuroSAT	97.8	**99.1**	+1.3
RESISC45	92.3	**96.5**	+4.2
fMoW	52.4	**58.9**	+6.5
Sen12Flood	84.2	**85.2**	+1.0
Cal. Wildfires	93.5	**94.9**	+1.4
BigEarthNet	82.6	**88.7**	+6.1
DFC2020	47.9	**54.3**	+6.4
Spacenet1	75.8	**78.2**	+2.4
Sen1Floods11	85.2	**89.6**	+4.4
DIOR	73.6	**79.0**	+5.4

Q3: Does GeoSANE go beyond model merging (is it just interpolating weights?)

Q5: How does GeoSANE improve and generalize across used model prompts?

WeightCLIP

A. Asefaw, K. Tzevelekakis, D. Falk, L. Meynent, D. Borth
WeightCLIP: Aligning Datasets and Models for Weight Space Learning
International Conference on Machine Learning (ICML), 2026



WeightCLIP: Aligning Datasets and Models for Weight Space Learning

Aron Asefaw [1] Konstantinos Tzevelekakis [1] Damian Falk [1] Léo Meynent [1] Damian Borth [1]

Abstract

Weight space learning aims to learn representations of neural network (NN) weights, enabling different downstream tasks. Existing approaches show promising performance, but lacking a way to shape these weight-space representations using information about the datasets the models were trained on, thus limiting downstream applications. We propose **WeightCLIP**, a method for learning a dataset-aligned latent space for neural networks, where datasets information is induced during training. The NNs are encoded as latent representations using an autoencoder, while dataset samples are encoded using a dataset encoder. The two representations are aligned using a contrastive objective, effectively reshaping the weight-space representations according to the datasets. We demonstrate that such representations can be used for different downstream tasks, including mapping dataset information to a weight-space representation that decode to strong models. In addition, we introduce a latent refinement process for generating models that outperforms standard fine-tuning. Overall, our results demonstrate that explicitly incorporating dataset information improves what can be achieved with weight-space representations across retrieval, generation, and refinement. Code will be available at github.com/HSG-AIML/WeightCLIP.

1. Introduction

In recent years, a new training paradigm coined weight space learning (WSL) has emerged that treats the parameters of neural networks as a data modality for representation learning. Once such representations are learned from a population of neural networks (referred to as model zoos),

[1]School of Computer Science, University of St.Gallen, Switzerland. Correspondence to: Aron Asefaw <aron.asefaw@unisg.ch>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

Figure 1. WeightCLIP trains a latent representation of model weights using a contrastive loss against the dataset the model weights have been trained on. Once trained, a *data prompt* for an unseen dataset can be used to generate model weights specifically for this dataset.

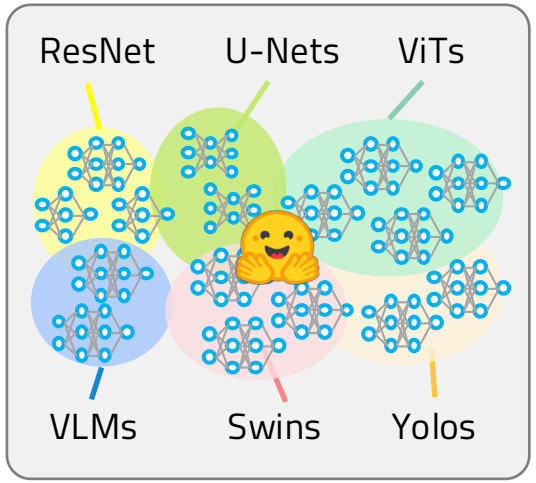
they can be used for several downstream tasks such as the prediction of model properties from weights (Unterthiner et al., 2020; Eilertsen et al., 2020; Martin & Mahoney, 2021; Schürholt et al., 2021; 2024; Navon et al., 2023; Zhou et al., 2023a), or the synthesis of unseen weights to generate entire models (Schürholt et al., 2022a; 2024; Knyazev et al., 2023; 2025; Kofinas et al., 2024; Wang et al., 2024; 2025; Bedionita et al., 2025b;a; Falk et al., 2025; Liang et al., 2026).

Much like NLP and CV benefited from training on large corpora, WSL has gained substantially from scaling to larger and more diverse model populations (Knyazev et al., 2023; Schürholt et al., 2024; Wang et al., 2024; 2025; Bedionita et al., 2025b;a; Falk et al., 2025). However, while modern NLP and CV increasingly rely on cross-modal supervision (e.g., text–image alignment) to endow representations with shared semantics, weight-space representations often lack an explicit *semantic reference frame* that makes learned representations intuitive to prompt, navigate, and explore. A key reason is that the geometry induced by training is not well-behaved. Model weights do contain information about the data distribution that shaped them, but this signal competes with confounders such as optimization noise, hyperparameter variation, and weight symmetries. As a result, latent spaces learned purely for reconstruction (Schürholt et al., 2022a; 2024; Bedionita et al., 2025b;a) or property prediction (Unterthiner et al., 2020; Eilertsen et al., 2020; Martin & Mahoney, 2021; Schürholt et al., 2021; 2024; Navon et al.,

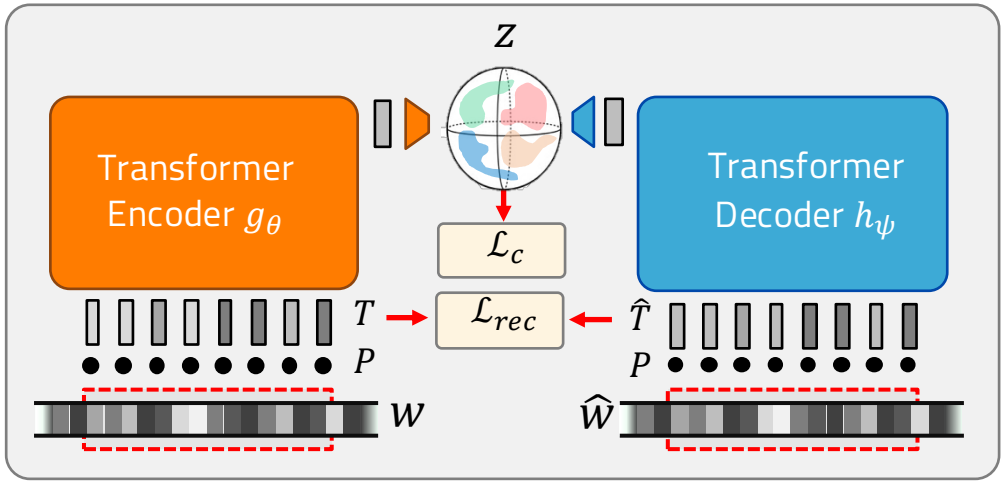
1

arXiv:2607.03551v1 [cs.LG] 3 Jul 2026

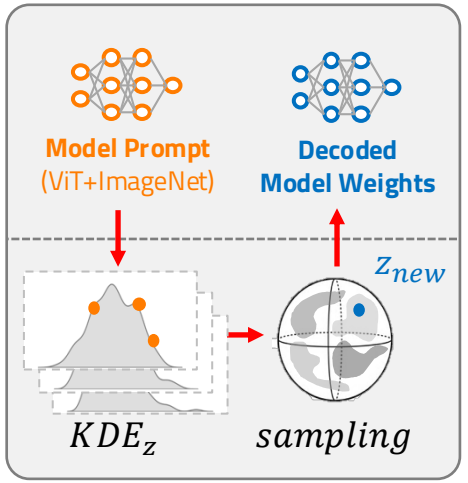
GeoSANE Setup



Model Zoo (Hugging Face)



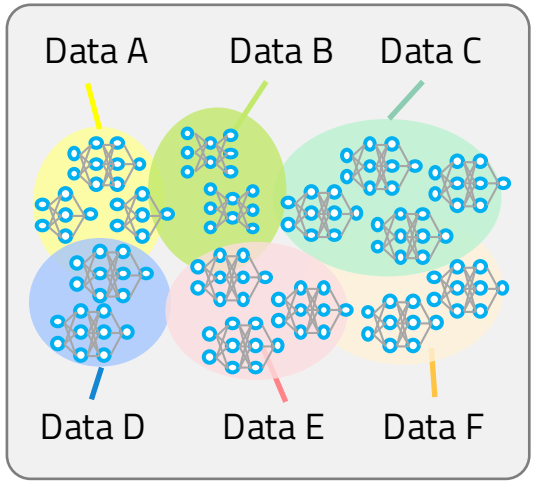
Representation Learning Backbone (SANE)



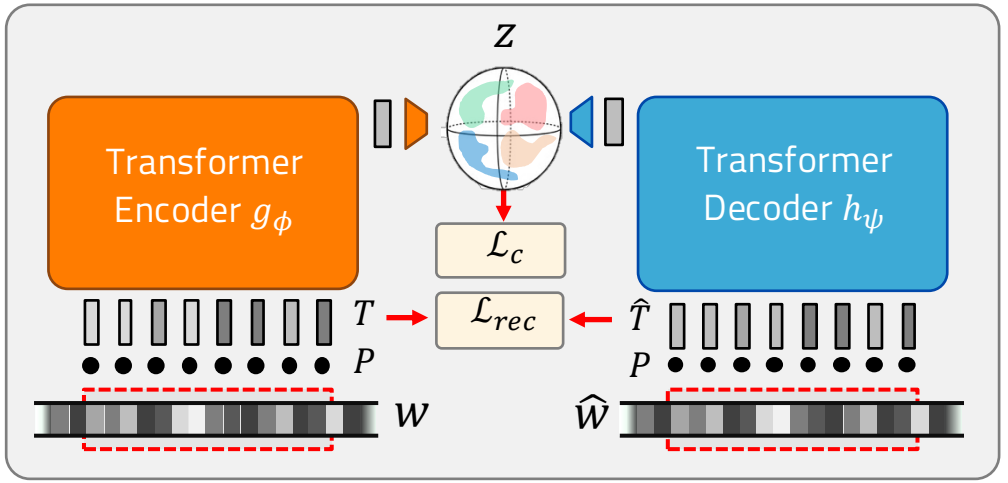
Model Generation

WeightCLIP Setup

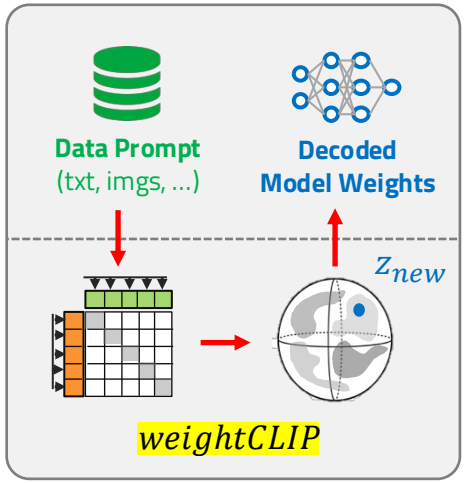
CNNs / ResNet



Curated Model Zoos

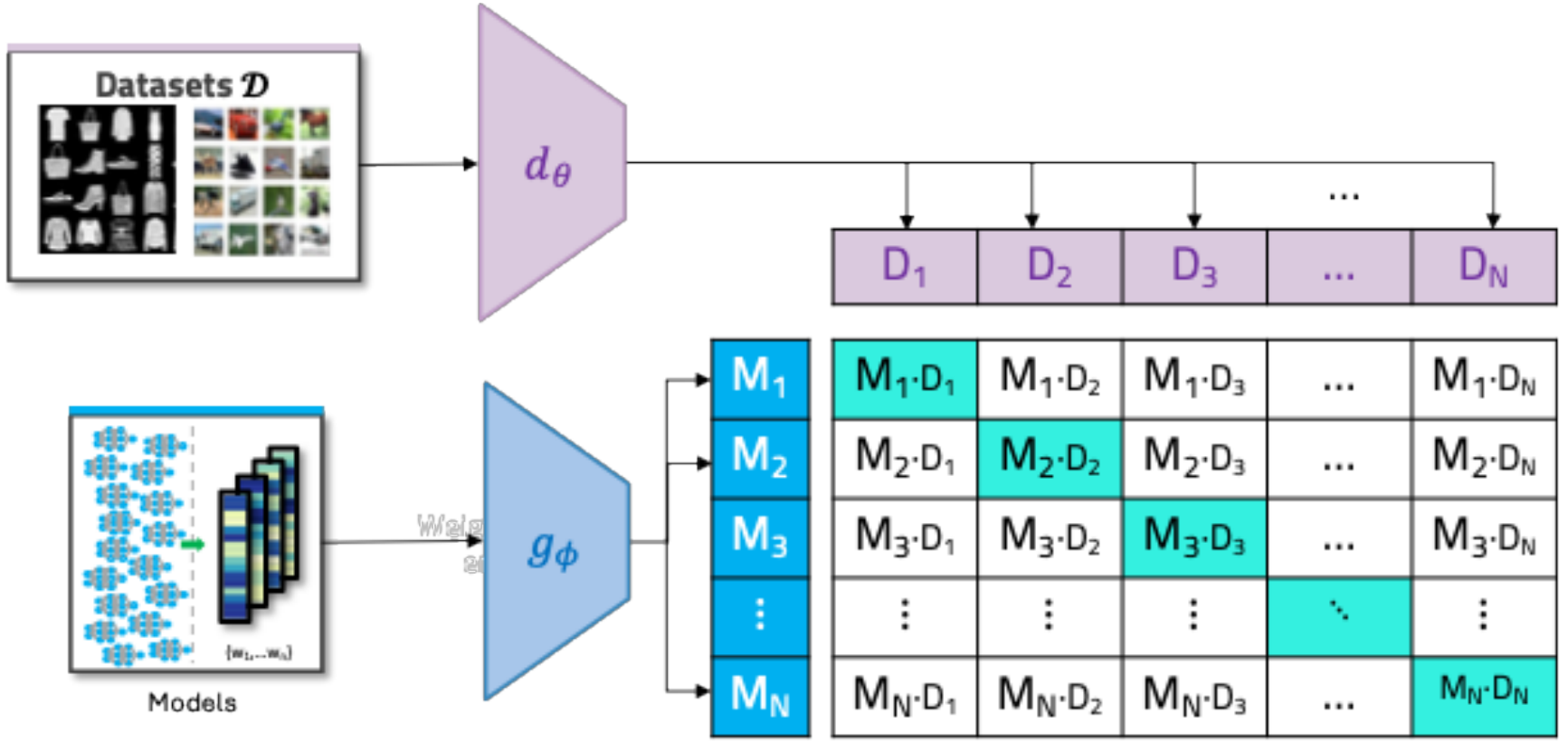


Representation Learning Backbone (SANE)

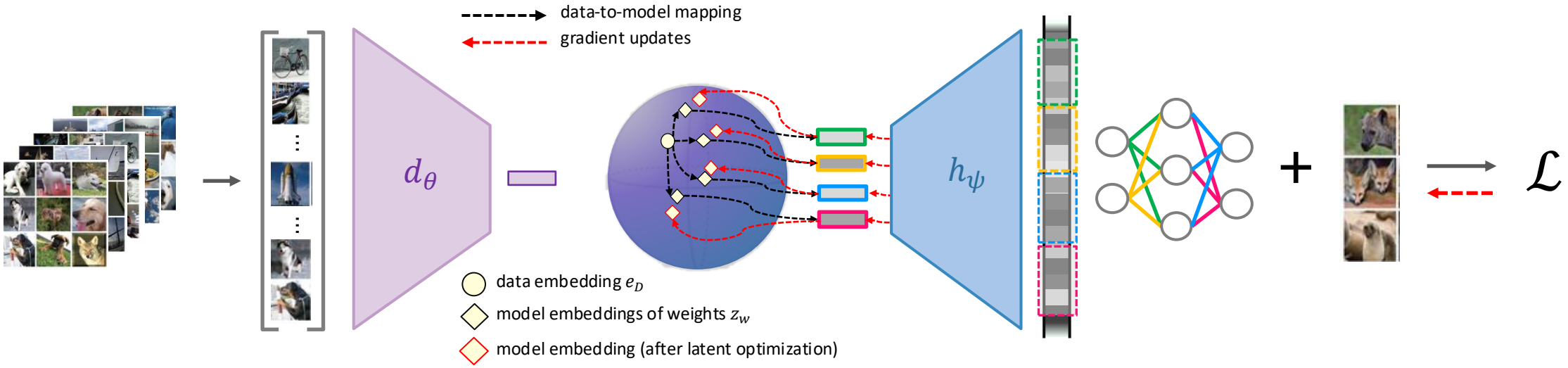


Model Generation

CLIP for Neural Network Models and Datasets



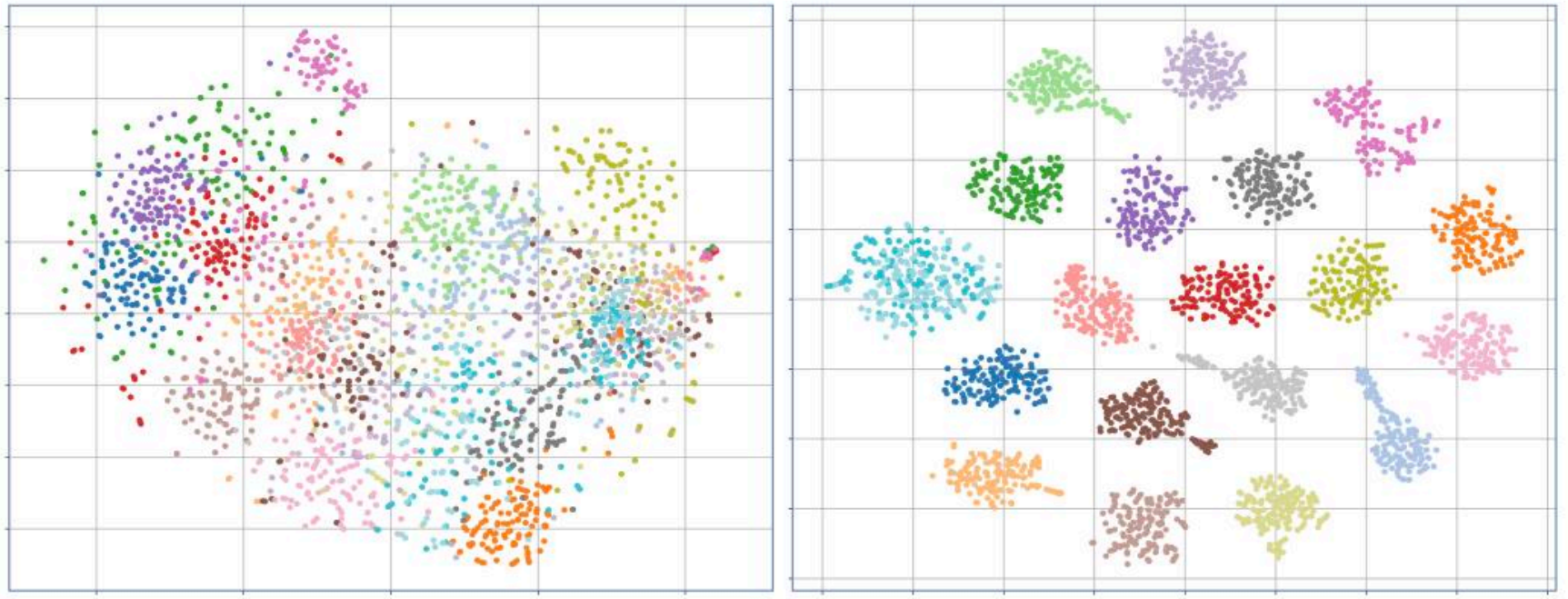
Prompt your Neural Network Model with Data



D → $x \subseteq D$ → $e_D = d_\theta(x)$ → $\{z_i\}^m = mp(e_D)$ → $\{w_i\}^m = h_\psi(\{z_i\}^m)$

dataset | subset of data | data embedding | model embedding of weights | decoded weights

WeightCLIP Organizes Weight Latents by Dataset



Without alignment, model embeddings from different datasets overlap. With WeightCLIP, models trained on the same dataset form compact clusters

OOD Dataset Prompts Generate Strong Initializations

ResNets-18

Ep.	Method	Colo.H	COVID-19	CIFAR10	Speed.	Honey P.	Real/Draw.
	tr. fr. scratch	∼12.5	∼33.3	∼10.0	∼25.0	∼50.0	∼10.0
	TANS	8.5±6.5	44.9±0.9	8.6±1.8	30.6±2.8	51.9±1.6	10.3±1.3
0	Hypernetwork	12.4±0.0	35.6±0.0	9.5±0.0	13.9±0.0	**59.7±0.0**	12.6±0.0
	WeightCLIP-LM	13.7±0.3	**47.4±0.4**	10.8±0.1	36.7±3.2	56.9±1.2	16.2±0.9
	WeightCLIP-MBM	**18.3±1.8**	45.9±0.3	**11.0±0.2**	**40.0±1.0**	52.6±1.0	**18.1±0.2**

Given an OOD dataset prompt, WeightCLIP predicts a ResNet18 model-latent and decodes it into model weights.

OOD Dataset Prompts Generate Strong Initializations

ResNets-18

Ep.	Method	Colo.H	COVID-19	CIFAR10	Speed.	Honey P.	Real/Draw.
0	tr. fr. scratch	∼12.5	∼33.3	∼10.0	∼25.0	∼50.0	∼10.0
	TANS	8.5±6.5	44.9±0.9	8.6±1.8	30.6±2.8	51.9±1.6	10.3±1.3
	Hypernetwork	12.4±0.0	35.6±0.0	9.5±0.0	13.9±0.0	**59.7±0.0**	12.6±0.0
	WeightCLIP-LM	13.7±0.3	**47.4±0.4**	10.8±0.1	36.7±3.2	56.9±1.2	16.2±0.9
	WeightCLIP-MBM	**18.3±1.8**	45.9±0.3	**11.0±0.2**	**40.0±1.0**	52.6±1.0	**18.1±0.2**
1	tr. fr. scratch	54.1±1.0	60.7±4.1	38.9±0.6	26.1±6.2	56.4±1.9	27.0±1.2
	TANS	29.2±9.3	73.5±1.9	41.5±0.3	32.4±1.6	60.7±7.7	21.9±2.8
	Hypernetwork	59.1±0.5	82.9±0.5	42.7±0.4	21.1±1.0	82.3±3.1	34.2±0.3
	WeightCLIP-LM	**68.4±4.6**	84.7±1.4	46.2±3.1	41.7±6.8	**82.8±6.4**	**35.9±2.7**
	WeightCLIP-MBM	68.4±3.8	**90.8±0.6**	**47.7±3.5**	**48.0±2.6**	72.9±3.7	35.4±1.7

Given an OOD dataset prompt, WeightCLIP predicts a ResNet18 model-latent and decodes it into model weights.

After short fine-tuning, WeightCLIP gives stronger initializations than scratch, retrieval, and hypernetwork baselines.

Latent Refinement Improves Over Fine-Tuning

CNNS

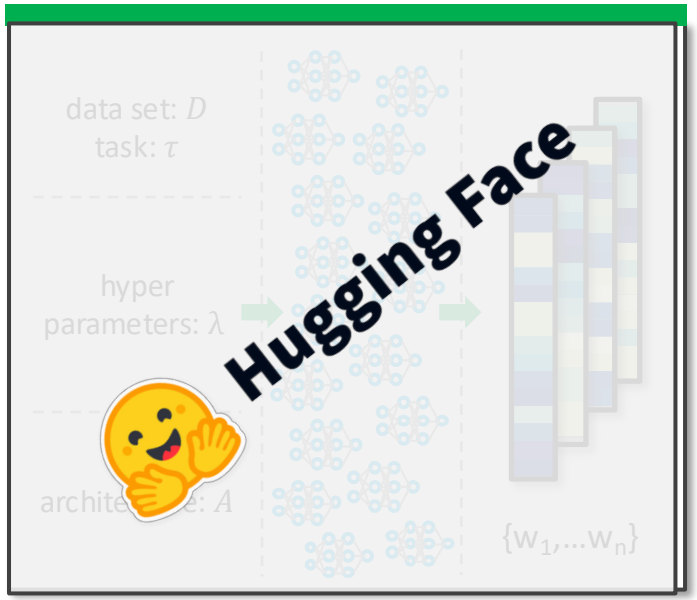
Dataset	Init	Fine-tune	Latent Refine	Δ
Colo.H	23.9±0.1	36.7±0.9	**37.0±0.6**	+0.3
COVID-19	46.3±0.8	57.3±1.3	**65.6±2.0**	+8.3
Speed. Signs	28.7±3.5	48.2±1.3	**60.2±5.7**	+12.0
CIFAR10	10.2±1.1	16.7±1.4	**19.5±1.7**	+2.8
Honey P.	46.3±0.7	54.6±0.7	**62.0±5.1**	+7.4
Real/Draw.	10.7±0.6	19.2±1.6	**24.9±2.0**	+5.7
Average	27.7±1.1	38.8±1.2	**44.9±2.9**	**+6.1**

WeightCLIP also introduces an alternative to standard fine-tuning: performing gradient steps in the latent space of weights!

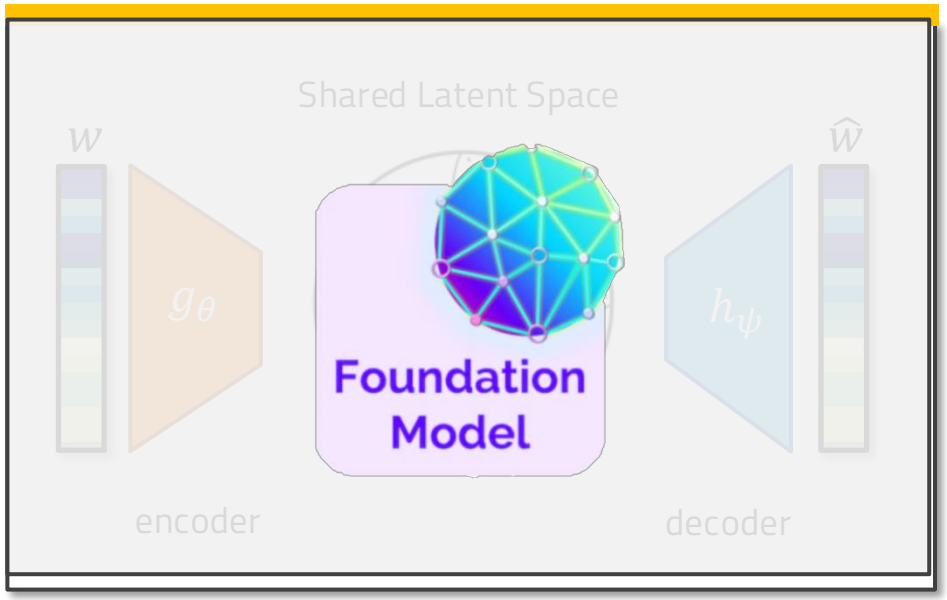
Under the same budget (1000 steps), this latent refinement improves over standard weight-space fine-tuning.

What are the current Challenges in Weight Space Learning?

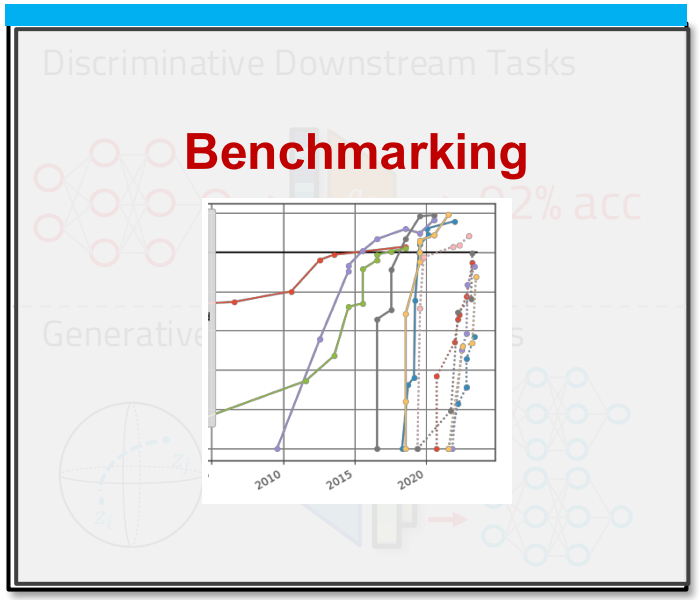
Three Challenges in Weight Space Learning



(A) Model Zoos



(B) Representation Learning Backbone



(C) Downstream Tasks



Discriminative Hyper-Reps NeurIPS 2021



Generative Hyper-Reps NeurIPS 2022



Model Zoos NeurIPS 2022



SANE ICML 2024



Phase-Transition Model Zoo DMLR 2026



GeoSANE CVPR 2026



WeightCLIP ICML 2026

Questions?

SANE: https://github.com/HSG-AIML/SANE
GeoSANE: https://github.com/HSG-AIML/GeoSANE
WeightCLIP: https://github.com/HSG-AIML/WeightCLIP

Survey: "A survey of weight space learning: Understanding, representation, and generation", 2026